

INAMANIA

Segundo de Desarrollo de Aplicaciones Web

Iván Martínez Navarro

15/05/2026

HOJA 0

Enlace a github: <https://github.com/martineznavarroivan26/Proyecto-intermodular>

Enlace a la web donde se ha desplegado la aplicación: <https://inamania.es>

Enlace a github del pdf de la memoria:

<https://github.com/martineznavarroivan26/Proyecto-intermodular/blob/main/docs/Proyecto%20Intermodular%20DAW%20Ivan%20Martinez%20Navarro.pdf>

Enlace a github del pdf del manual de usuario de la aplicación:

[https://github.com/martineznavarroivan26/Proyecto-intermodular/blob/main/docs/Manual Usuario Inamania v1.0.pdf](https://github.com/martineznavarroivan26/Proyecto-intermodular/blob/main/docs/Manual%20Usuario%20Inamania%20v1.0.pdf)

Usuarios creados para el uso de la aplicación:

1. Usuario tipo A
 - a. Usuario: admin@inamania.test
 - b. Contraseña: 123456789
2. Usuario tipo B
 - a. Usuario: mod@inamania.test
 - b. Contraseña: 123456789
3. Usuario tipo C
 - c. Usuario: raul@inamania.test
 - d. Contraseña: 123456789

Enlace a github de los scripts para crear e inicializar la base de datos:

<https://github.com/martineznavarroivan26/Proyecto-intermodular/blob/main/config/BD.sql>

1. Descripción del proyecto	6
1.1. Características generales	7
1.2. Objetivos y alcance	8
2. Análisis del sector / mercado	9
2.1. Prospectiva del Título en el sector.....	9
2.2. Evolución y tendencia del sector.....	9
2.3. Normativa y documentación técnica específica.....	9
3. Plan de ejecución.....	9
3.1. Diagrama/cronograma de flujo de procesos (Diagrama de Gantt).....	9
3.2. Proceso de desarrollo software.....	9
3.2.1. Fase de análisis.....	10
3.2.1.1. Tipos de usuario.....	11
3.2.1.2. Descripción de requisitos.....	12
3.2.1.3. Casos de Uso	13
3.2.1.4. Guía de estilo	17
3.2.1.5. Prototipo del sitio web	20
3.2.1.6. Mapa de navegación	21
3.2.2. Fase de desarrollo	22
3.2.2.1. Base de datos	23
3.2.2.1.1. Análisis de requisitos de datos de la aplicación.....	23
a) Identificación de entidades e interrelaciones.....	23
b) Identificación de atributos de cada entidad	24
3.2.2.1.2 Diseño lógico de datos	25
3.2.2.1.3 Paso del modelo lógico (E/R) al modelo relacional (tablas)	26
a. Identificación de atributos de cada tabla, sus tipos y tamaños	26
b. Identificación de claves principales y claves candidatas.....	29
c. Identificación de las reglas de integridad y seguridad de la base de datos.....	29
d. Modelo relacional (tablas)	30
3.2.2.1.4 Aplicación de reglas de normalización al modelo relacional....	32
3.2.2.1.5 Tipos de datos para el sistema gestor seleccionado	32

3.2.2.1.6 Scripts de creación de tablas e inserciones iniciales	33
3.2.2.2. Servidor	37
3.2.2.2.1. Lista de funciones en php	37
3.2.2.3. Cliente.....	37
3.2.2.3.1. Diseño de la interfaz	37
3.2.2.3.2. Accesibilidad	37
3.2.2.3.3. Usabilidad	37
3.2.2.3.4. Desarrollo web entorno cliente	37
a. Formularios y su validación.....	37
b. Manejo y gestión de eventos: Teclado, ratón y estados de la ventana	38
c. Gestión y almacenamiento de datos e información en el cliente	38
d. Modificación del DOM	38
e. Animaciones, efectos, cambios dinámicos de estilos etc... para dinamizar la parte visible al cliente.....	38
f. Comunicación AJAX	38
g. Comunicación asíncrona con el servidor	38
3.2.3. Fase de despliegue.....	38
3.2.3.1. Despliegue utilizando un hosting ó	38
3.2.3.2. Despliegue en un equipo local propio	38
3.3. Seguimiento y control de incidencias	39
3.4. Indicadores de calidad de procesos	39
4. Recursos materiales.....	39
4.1. Inventario, valorado de medios	39
4.2. Presupuesto económico	39
5. Recursos humanos.....	39
5.1. Organización	39
5.2. Contratación	40
5.3. Prevención de riesgos laborales	40
6. Viabilidad técnica.....	40
6.1. Estudio de viabilidad técnica	40
7. Viabilidad económica-financiera	40

7.1. Inversiones y gastos	40
7.2. Financiación	40
7.3. Viabilidad económico-financiera.....	41
8. Conclusión	41
Bibliografía/Webgrafía	41
ANEXOS.....	41

1. Descripción del proyecto

Este proyecto consiste en el desarrollo de una página web multifuncional centrada en el universo de Inazuma Eleven. La web servirá como guía completa de los videojuegos de la saga y, al mismo tiempo, como un espacio de comunidad interactivo.

La estructura estará pensada para que los usuarios encuentren fácilmente información de los juegos, pero también puedan interactuar con otros fans mediante posts, comentarios y quedadas. La prioridad será diseñar un portal atractivo, rápido y responsivo, con funcionalidades que combinen bases de datos dinámicas y herramientas sociales.

1.1. Características generales

El proyecto va a constar de una página web en la que reunamos toda la información y comunidad dispersa de inazuma eleven, que

- Crear una web responsive, accesible desde ordenador, tablet y móvil.
- Integrar bases de datos interactivas para guías y recursos de los juegos.
- Desarrollar un sistema de comunidad online con foros, posts y calendario de eventos.
- Implementar una navegación sencilla y confiable.

1.2. Objetivos y alcance

La finalidad es consolidar un punto de referencia confiable y atractivo para jugadores y seguidores, optimizado tanto para el acceso rápido a información como para la socialización dentro de la comunidad, con los objetivos de:

- **Diseño y experiencia de usuario:** Desarrollar una interfaz clara, atractiva e inspirada en la estética de Inazuma Eleven, adaptable a dispositivos móviles, tablets y PC.
- **Guías estructuradas:** Implementar un sistema organizado de guías de videojuegos con buscadores y filtros avanzados que permitan encontrar rápidamente personajes, equipos o técnicas específicas.
- **Funcionalidades sociales:** Incluir herramientas de interacción como creación de posts, comentarios y favoritos, fomentando la participación y cohesión de la comunidad.
- **Eventos y quedadas:** Incorporar un calendario dinámico que permita la organización de torneos, directos y encuentros, con opción de inscripciones online a eventos gratuitos y de pago.
- **Moderación y seguridad:** Implementar un panel de administración que permita controlar publicaciones, usuarios y reportes, garantizando un entorno seguro y agradable para todas las edades.
- **Mantenimiento:** Diseñar la web de forma modular para que pueda ampliarse con nuevas funcionalidades, contenidos o idiomas en el futuro.

El alcance del proyecto se centra en el desarrollo e implementación de la página web con las siguientes características clave.

Secciones principales de la web:

- **Inicio:** Noticias destacadas, novedades de la comunidad y eventos próximos.
- **Juegos:** Bases de datos dinámicas con personajes, equipos, técnicas, walkthroughs y coleccionables.
- **Foro:** Espacio de foros y posts divididos en categorías (estrategias, fanart, noticias, teorías, dudas).
- **Eventos:** Calendario interactivo con inscripciones, recordatorios y ranking de torneos.
- **Perfil:** Espacio personalizable para cada usuario, con avatar, favoritos, logros y amigos.
- **Usuarios objetivo:** Jugadores y fans de todas las edades, desde principiantes que buscan guías, hasta comunidades organizadas interesadas en torneos y quedadas.
- **Seguridad:** Un sistema de seguridad que pueda proteger al usuario y sus datos personales.

2. Análisis del sector / mercado

Antes de desarrollar el proyecto es necesario realizar un análisis del sector y del mercado en el que va a operar. Este estudio permite conocer la situación actual de la industria, identificar las oportunidades existentes, analizar a la competencia y determinar las necesidades reales de los potenciales usuarios.

Inamania se desarrolla dentro del sector de los servicios digitales relacionados con los videojuegos y las comunidades online. Debido al crecimiento constante de la industria del entretenimiento digital y al aumento de usuarios que buscan información especializada, resulta fundamental estudiar las tendencias del mercado y evaluar la viabilidad de una plataforma centrada en la saga Inazuma Eleven.

A través de este análisis se pretende identificar el público objetivo del proyecto, las características del sector, la evolución prevista del mercado, la competencia existente y la normativa aplicable, con el fin de garantizar que la propuesta responda a una necesidad real y disponga de posibilidades de crecimiento a medio y largo plazo.

2.1. Prospectiva del Título en el sector

En la actualidad, el sector de los videojuegos constituye una de las industrias de entretenimiento con mayor crecimiento a nivel mundial. Paralelamente, han surgido numerosas plataformas digitales especializadas que permiten a los usuarios acceder a información, compartir experiencias y participar en comunidades relacionadas con sus videojuegos favoritos.

Dentro de este contexto se sitúa Inamania, una plataforma especializada en la franquicia Inazuma Eleven que pretende centralizar información sobre juegos, personajes, equipos, técnicas especiales y eventos relacionados con la saga. La especialización temática permite ofrecer contenidos más completos y organizados que los disponibles en plataformas generalistas.

El perfil profesional asociado al ciclo formativo de Desarrollo de Aplicaciones Web presenta una elevada demanda dentro de este sector, ya que las empresas y proyectos digitales requieren profesionales capaces de diseñar, desarrollar y mantener aplicaciones web modernas, seguras y accesibles. Las competencias adquiridas durante el ciclo permiten abordar todas las fases necesarias para la construcción de una plataforma como Inamania, desde el diseño de bases de datos hasta la implementación de interfaces de usuario y sistemas de gestión de contenidos.

2.2. Evolución y tendencia del sector

La evolución del sector muestra una creciente importancia de las comunidades digitales y del contenido generado por los propios usuarios. Los jugadores ya no solo buscan información sobre videojuegos, sino también espacios donde interactuar con otros usuarios, compartir experiencias y participar en actividades relacionadas con sus intereses.

Entre las principales tendencias del sector destacan:

- Incremento del consumo de contenidos digitales relacionados con videojuegos.
- Crecimiento de las comunidades online especializadas.
- Mayor utilización de dispositivos móviles para acceder a servicios web.
- Integración de funcionalidades sociales como perfiles de usuario, comentarios y sistemas de participación.
- Organización de eventos presenciales y virtuales para fortalecer las comunidades.
- Desarrollo de plataformas especializadas que ofrecen información más completa y estructurada que las redes sociales tradicionales.

La publicación de nuevos títulos de la saga Inazuma Eleven y la continuidad de su comunidad de seguidores favorecen la existencia de proyectos especializados como Inamania, proporcionando un nicho de mercado con potencial de crecimiento.

2.3. Normativa y documentación técnica específica

Para el desarrollo y funcionamiento de Inamania será necesario cumplir con la legislación vigente aplicable a los servicios digitales y a la protección de datos personales.

1. Reglamento General de Protección de Datos (RGPD)

Establece las obligaciones relacionadas con el tratamiento de datos personales de los usuarios, garantizando la privacidad y la seguridad de la información almacenada.

2. Ley Orgánica de Protección de Datos Personales y Garantía de los Derechos Digitales (LOPDGDD)

Complementa la normativa europea y regula la protección de datos en el ámbito nacional.

3. Ley de Servicios de la Sociedad de la Información y Comercio Electrónico (LSSI-CE)

Regula las obligaciones legales de los prestadores de servicios digitales, incluyendo la información que debe ofrecerse a los usuarios y las comunicaciones electrónicas.

4. Propiedad intelectual y derechos de autor

La plataforma respetará los derechos de propiedad intelectual asociados a la franquicia Inazuma Eleven y a cualquier contenido protegido utilizado con fines informativos.

5. Accesibilidad web

Durante el desarrollo se tendrán en cuenta las recomendaciones WCAG (Web Content Accessibility Guidelines) para garantizar una experiencia accesible para todos los usuarios.

6. Documentación técnica del proyecto

La documentación técnica incluirá:

- Análisis de requisitos.
- Diseño de la base de datos.
- Modelo entidad-relación.
- Modelo relacional.
- Diseño de interfaces.
- Manual técnico.
- Manual de usuario.
- Documentación del código fuente.

Así mantienes exactamente la estructura que ya tiene el proyecto y además metes todo el análisis de mercado sin inventarte apartados nuevos.

3. Plan de ejecución

Para llevar este portal de la idea a la realidad en 2026, he organizado el trabajo en cuatro fases críticas. Mi prioridad es lanzar un producto funcional lo antes posible y luego ir escalando según responda la comunidad.

Fase 1: Cimientos y Base de Datos

En esta etapa me centraré en lo que hace que la web sea útil desde el primer día.

- **Arquitectura:** Configuración del servidor y la base de datos (MySQL).
- **El "Motor de Fichajes":** Desarrollo del sistema de filtrado para personajes, técnicas y objetos. Es el trabajo más pesado: volcar y organizar la información de los juegos.
- **Diseño UI/UX:** Creación de los prototipos visuales con la estética de *Inazuma Eleven* (colores eléctricos, menús dinámicos).

Fase 2: El Núcleo del Usuario

Aquí es donde la web deja de ser un libro de consulta y empieza a ser una plataforma.

- **Sistema de Cuentas:** Registro, inicio de sesión seguro y personalización del perfil ("Carnet de Jugador").
- **Panel de Administración:** Construcción de las herramientas internas para que yo y mis moderadores podamos gestionar el contenido y los usuarios.
- **SEO Técnico:** Optimización para que, cuando alguien busque "Mejores delanteros Victory Road", mi web aparezca la primera.

Fase 3: Despliegue de Comunidad

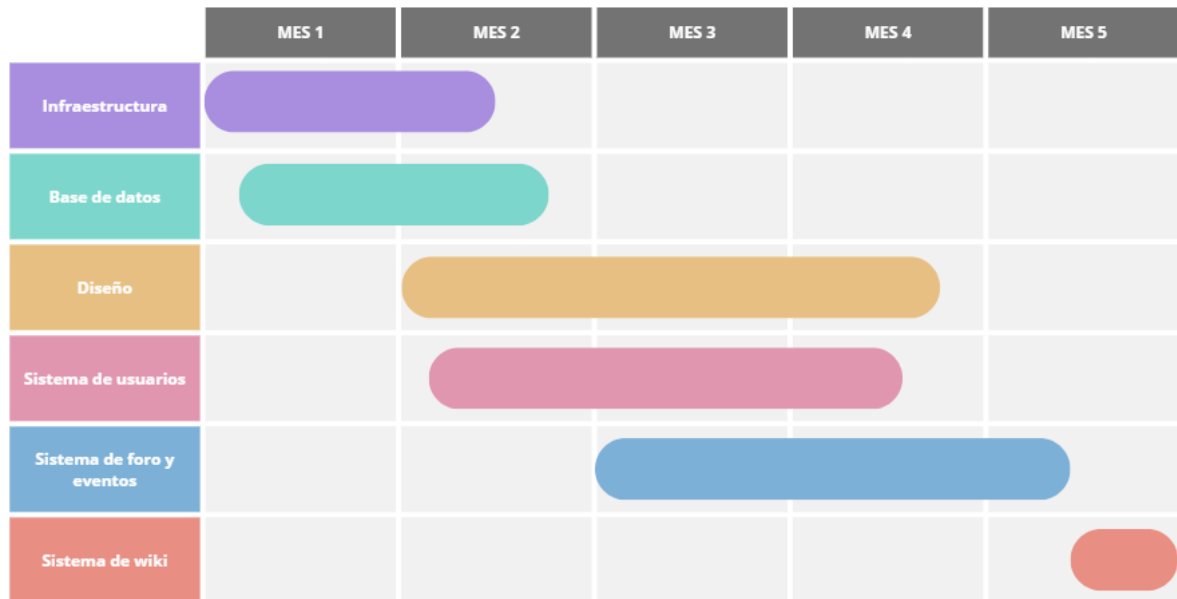
Con la base de datos lista, abro las funciones sociales.

- **Foros y Comentarios:** Activación de los hilos de discusión por categorías (estrategia, fanart, etc.).
- **Módulo de Eventos:** Lanzamiento del calendario interactivo donde empezaré a registrar los primeros torneos y quedadas de la comunidad.
- **Beta Cerrada:** Invitación a un grupo pequeño de fans para que testeen la web y encontrar fallos antes del estreno oficial.

Fase 4: Lanzamiento y Escalabilidad

- **Gran Estreno:** Apertura oficial del portal y campaña en redes sociales (Twitter/X, TikTok).
- **Mantenimiento:** Actualización semanal con los nuevos datos que Level-5 vaya lanzando en sus parches.

3.1. Diagrama/cronograma de flujo de procesos (Diagrama de Gantt)



3.2. Proceso de desarrollo software

He decidido adoptar una metodología **Ágil (Scrum)**. Esto significa que no voy a esperar 5 meses para ver resultados; voy a trabajar en "Sprints" (ciclos cortos de 2 semanas) para ir lanzando partes funcionales de la web.

Fase A: Análisis y Diseño de Requisitos

Antes de tocar el teclado, voy a definir las **Historias de Usuario**. Por ejemplo: *"Como usuario, quiero poder filtrar jugadores por su elemento para montar mi equipo rápidamente"*. Esto me ayuda a que cada línea de código tenga un objetivo real para el fan.

Fase B: Arquitectura y Stack Tecnológico

Voy a montar el proyecto usando un stack moderno que me garantice velocidad:

- **Frontend:** con html, css y js puros ayudándonos un poco de bootstrap (para que las guías de los juegos carguen instantáneamente).
- **Backend:** php con una base de datos MySQL (ideal para manejar las miles de combinaciones de técnicas y jugadores).

Fase C: Implementación (Ciclo de Desarrollo)

Aquí es donde ocurre la magia. Dividiré el desarrollo en tres capas:

- **Capa de Datos:** Crear la estructura donde vivirán los personajes y equipos.
- **Capa Lógica:** Programar los buscadores, los sistemas de registro y el calendario de eventos.
- **Capa Visual:** Aplicar el estilo visual de *Inazuma Eleven* para que la experiencia sea inmersiva.

Fase D: Pruebas y Control de Calidad (QA)

No quiero errores el día del estreno. Por lo que realizare:

- **Pruebas Unitarias:** Para asegurar que los filtros de búsqueda funcionan bien.

Fase E: Despliegue y Mantenimiento

Usaré un sistema de **Despliegue Continuo (CI/CD)**. Esto significa que cualquier mejora o corrección que haga en el código se subirá automáticamente a la web sin necesidad de cortarla, permitiendo que el portal esté siempre online y actualizado con los últimos parches de *Victory Road*.

3.2.1. Fase de analisis

En esta fase de análisis he sintetizado las necesidades clave para asegurar que el proyecto sea un éxito rotundo:

- **Identificación del problema:** La información actual está fragmentada en Discord y wikis obsoletas. Mi oportunidad es centralizarla en una web visual y rápida.
- **Historias de usuario:** He definido que mis usuarios buscan tres cosas: optimización (filtros de stats), guías claras (fichajes paso a paso) y conexión (inscripción a torneos).
- **Viabilidad técnica y legal:** He confirmado que (PHP/MySQL) es ideal para grandes bases de datos y he blindado el proyecto con normativas de RGPD y un enfoque de Fair Use ante Level-5.
- **Estructura del sitio:** He diseñado una arquitectura donde el contenido (guías) y la comunidad (foro/eventos) conviven de forma intuitiva y accesible.

3.2.1.1. Tipos de usuario

Usuario visitante (no registrado):

Descripción: Usuario que accede a la página sin crear una cuenta.

Permisos: Solo de lectura.

Funcionalidades:

- Navegar por la web y consultar información básica.
- Acceder a guías, walkthroughs, bases de datos de personajes y técnicas.
- Visualizar posts publicados en la comunidad.
- Consultar el calendario de eventos sin posibilidad de inscribirse.

Usuario registrado:

Descripción: Usuario que crea una cuenta y puede interactuar activamente en la comunidad.

Permisos: Lectura y escritura en las áreas sociales.

Funcionalidades:

- Acceso a todas las funciones de **usuario visitante**.
- Crear y editar posts en la comunidad.
- Comentar publicaciones.
- Guardar posts en favoritos.
- Personalizar su perfil (avatar, biografía).
- Inscribirse a eventos y quedadas.

Usuario Moderador:

Descripción: Usuario que modera organiza y controla el contenido del foro y eventos.

Permisos: Lectura, escritura en las áreas sociales, borrar mensajes, añadir eventos y noticias.

Funcionalidades:

- Crear eventos, modificarlos y borrarlos.
- Administrar comentarios de usuarios y perfiles.
- Añadir noticias.
- Actualizar la guía

Usuario Administrador:

Descripción: Usuario que modera organiza y controla el contenido del foro y eventos y ajusta los roles de usuarios.

Permisos: Lectura, escritura en las áreas sociales, borrar mensajes, añadir eventos y noticias.

Funcionalidades:

- Crear eventos, modificarlos y borrarlos.
- Administrar comentarios de usuarios y perfiles.
- Añadir noticias.
- Actualizar la guía.
- Ajustar roles de usuarios.

3.2.1.2. Descripción de requisitos

1. Gestión de usuarios

- El sistema deberá permitir el registro de nuevos usuarios mediante correo electrónico y contraseña.
- El sistema deberá permitir el inicio de sesión con correo
- El sistema deberá permitir la edición del perfil de usuario (nombre, avatar, biografía, contraseña).

2. Contenido de guías de los videojuegos

- El sistema deberá permitir consultar guías de cada juego de la saga Inazuma Eleven.
- El sistema deberá ofrecer una base de datos navegable de personajes, equipos, técnicas y torneos.
- El sistema deberá permitir realizar búsquedas y filtrados avanzados.

3. Comunidad

- El sistema deberá permitir a los usuarios registrados crear, editar y eliminar posts en la comunidad.
- El sistema deberá permitir a los usuarios registrados comentar publicaciones.
- El sistema deberá permitir reacciones a posts y comentarios (me gusta).
- El sistema deberá permitir guardar favoritos (posts, guías, personajes).

4. Eventos y quedadas

- El sistema deberá mostrar un calendario de eventos interactivo con fechas de torneos, quedadas o directos.
- El sistema deberá permitir a los usuarios registrados inscribirse en eventos.
- El sistema deberá enviar notificaciones y recordatorios a los inscritos antes del inicio de un evento.
- El sistema deberá mostrar información detallada de cada evento.

5. Interfaz

- Una navegación sencilla y amigable.
- Un diseño responsive que se adapte a PC, Tablet y móvil
- Un aspecto que no sea sobrecargado y atractivo visualmente

3.2.1.3. Casos de Uso

Identificador CU	CU_01	
Nombre CU	Registro	
Actores	Usuario visitante	
Descripción	Permite al visitante crear una cuenta en la plataforma para acceder a las funciones sociales.	
Prioridad	Alta (10)	
Precondición	El usuario no está autenticado.	
Datos de entrada	Nombre, correo, contraseña, confirmación de contraseña, imagen de perfil.	
Datos de salida	Confirmación de registro o mensaje de error.	
Secuencia Normal		
Usuario		Sistema
1.Usuario accede al formulario de registro.		
		2.Valida los datos introducidos.
		3.Crea el usuario y lo guarda en la base de datos.
		4.Muestra mensaje de confirmación y ofrece permanecer con la sesión iniciada.
Flujos alternativos		
Usuario		Sistema
5.El correo usado ya existe.		
		6.Muestra un error y solicita otro correo.
7.La confirmación de la contraseña y la contraseña no coinciden.		
		8. Muestra un error y solicita que sean iguales.
Postcondición	Usuario registrado en la base de datos.	
CU Relacionados		

Identificador CU	CU_02
Nombre CU	Iniciar sesión
Actores	Usuario visitante
Descripción	Permite al visitante iniciar sesión con sus credenciales.
Prioridad	Alta (9)
Precondición	El usuario ya registrado.
Datos de entrada	Correo y contraseña.
Datos de salida	Confirmación de inicio de sesión o mensaje de error.
Secuencia Normal	
Usuario	Sistema
1. Usuario accede al formulario de inicio de sesión.	
	2. Valida los datos introducidos.
	3. Inicia sesión en el perfil de los datos.
Flujos alternativos	
Usuario	Sistema
5. Las credenciales son incorrectas.	
	6. Muestra un error y solicita otras credenciales.
Postcondición	Sesión activa.
CU Relacionados	CU_01

Identificador CU	CU_03
Nombre CU	Gestión de usuarios y posts
Actores	Administrador
Descripción	Permite moderar contenido y usuarios.
Prioridad	Alta (9)
Precondición	Usuario autenticado con rol de administrador.
Datos de entrada	Acciones de moderación.
Datos de salida	Confirmación o mensaje de error.
Secuencia Normal	
Usuario	Sistema
1. Admin accede al panel de control.	
	2. Muestra lista de publicaciones y usuarios.
3. Admin selecciona acción (eliminar, editar, bloquear).	
	4. Ejecuta la acción y muestra confirmación.
Flujos alternativos	
Usuario	Sistema
Postcondición	Contenido moderado o usuario actualizado.
CU Relacionados	CU_05, CU_06, CU_07

Identificador CU	CU_04
Nombre CU	Cerrar sesión
Actores	Usuario registrado
Descripción	Permite al usuario registrado cerrar la sesión en el dispositivo.
Prioridad	Alta (8)
Precondición	El usuario tiene la sesión iniciada.
Datos de entrada	Confirmar cerrar sesión
Datos de salida	Confirmación de sesión cerrada o mensaje de error.
Secuencia Normal	
Usuario	Sistema
1. Usuario solicita cerrar sesión.	
	2. Confirmación de cerrar sesión.
	4. Sistema muestra mensaje de confirmación y te vuelve visitante.
Flujos alternativos	
Usuario	Sistema
5. No confirmas cerrar sesión.	
	6. Cierra la función de cerrar sesión.
Postcondición	
CU Relacionados	CU_01, CU_02
Identificador CU	CU_05
Nombre CU	Crear post en la comunidad
Actores	Usuario registrado
Descripción	Permite publicar un post en una categoría del foro.
Prioridad	Alta (7)
Precondición	El usuario con sesión iniciada.
Datos de entrada	Título, categoría, contenido, imagen opcional.
Datos de salida	Post guardado y visible en la comunidad.
Secuencia Normal	
Usuario	Sistema
1. Usuario accede a "Crear post".	
	2. Muestra formulario.
3. Usuario introduce datos y envía.	
	4. Valida y guarda post.
	5. Se muestra confirmación y post publicado.
Flujos alternativos	
Usuario	Sistema
6. Faltan datos obligatorios.	
	6. Muestra un error y solicita los datos.
Postcondición	Post visible en el foro.
CU Relacionados	CU_06, CU_03

Identificador CU	CU_06	
Nombre CU	Comentar post en la comunidad	
Actores	Usuario registrado	
Descripción	Permite comentar un post existente en la comunidad.	
Prioridad	Media (6)	
Precondición	El usuario con sesión iniciada.	
Datos de entrada	Comentario.	
Datos de salida	Comentario añadido al post.	
Secuencia Normal		
Usuario		Sistema
1. Usuario selecciona un post.		
		2. Muestra los comentarios existentes.
3. Usuario escribe y envía comentario.		
		4. Sistema guarda el comentario.
		5. Se muestra confirmación y se ve el comentario en el post.
Flujos alternativos		
Usuario		Sistema
6. Campo vacío.		
		6.Muestra un error y solicita los datos.
Postcondición	Comentario añadido.	
CU Relacionados	CU_05, CU_03	

Identificador CU	CU_07	
Nombre CU	Inscribirse a un evento	
Actores	Usuario registrado	
Descripción	Permite registrarse en un evento o torneo.	
Prioridad	Media (6)	
Precondición	El usuario con sesión iniciada y evento activo.	
Datos de entrada	Identificador de evento.	
Datos de salida	Confirmación de inscripción o mensaje de error.	
Secuencia Normal		
Usuario		Sistema
1. Usuario accede al calendario de eventos.		
2. Selecciona un evento.		
		3. Valida disponibilidad.
		4. Registra la inscripción.
		5. Se muestra confirmación.
Flujos alternativos		
Usuario		Sistema
Postcondición	Usuario inscrito en el evento.	
CU Relacionados	CU 03	

3.2.1.4. Guía de estilos

Colores

 #FF8C00	Color principal	Usado para elementos principales que resalten mucho
 #FFC773	Color secundario	Usado para los elementos principales de contenido
 #1E90FF	Color contraste	Usado para resaltar elementos
 #0050A0	Color hover	Usado para el hover del contraste

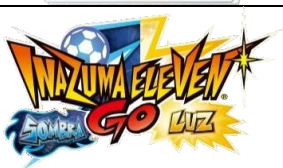
Tipografía

La tipografía que voy a usar es la Montserrat.

Thin 100	Hola esto es una prueba
Thin 100 italic	<i>Hola esto es una prueba</i>
ExtraLight 200	Hola esto es una prueba
ExtraLight 200 italic	<i>Hola esto es una prueba</i>
Light 300	Hola esto es una prueba
Light 300 italic	<i>Hola esto es una prueba</i>
Regular 400	Hola esto es una prueba
Regular 400 italic	<i>Hola esto es una prueba</i>
Medium 500	Hola esto es una prueba
Medium 500 italic	<i>Hola esto es una prueba</i>
SemiBold 600	Hola esto es una prueba
Bold 700	Hola esto es una prueba
Bold 700 italic	<i>Hola esto es una prueba</i>
ExtraBold 800	Hola esto es una prueba
ExtraBold 800 italic	<i>Hola esto es una prueba</i>
Black 900	Hola esto es una prueba
Black 900 italic	<i>Hola esto es una prueba</i>

Iconografía

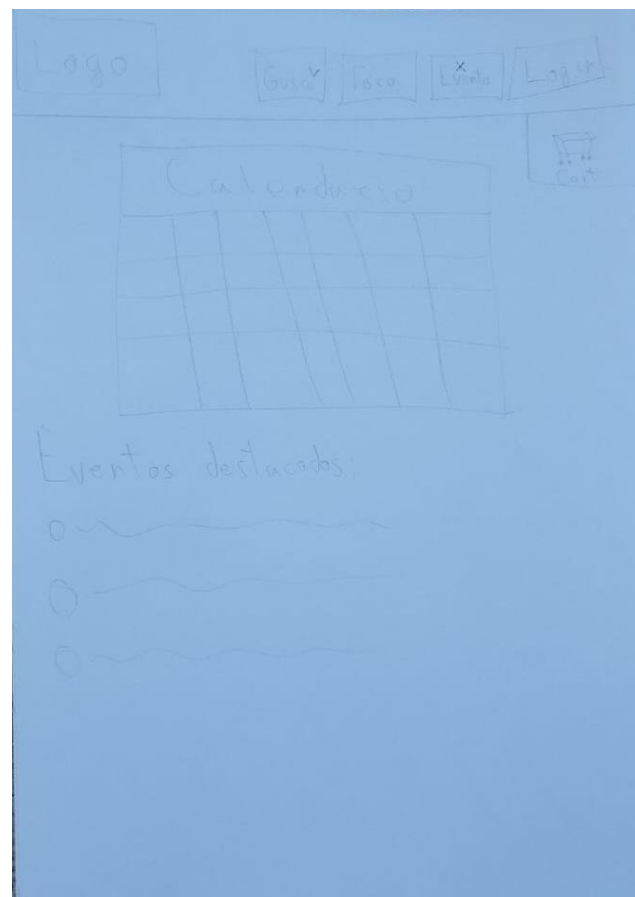
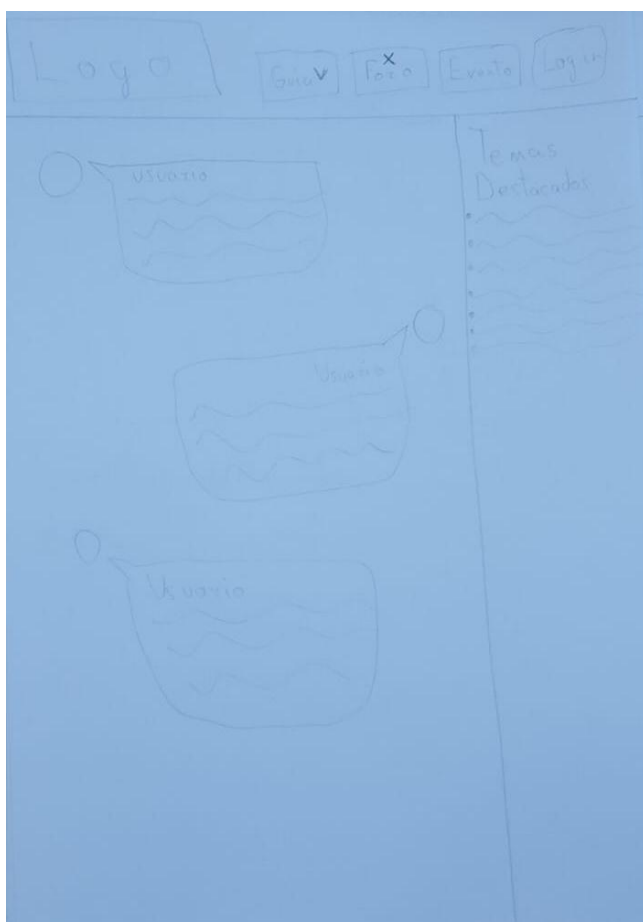
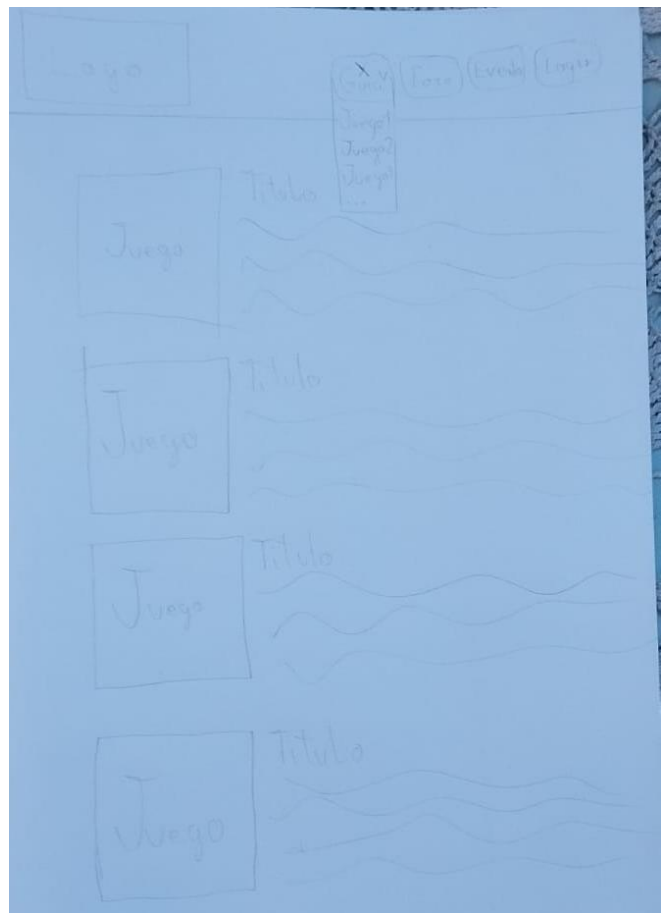
★ Indica que lo hice yo

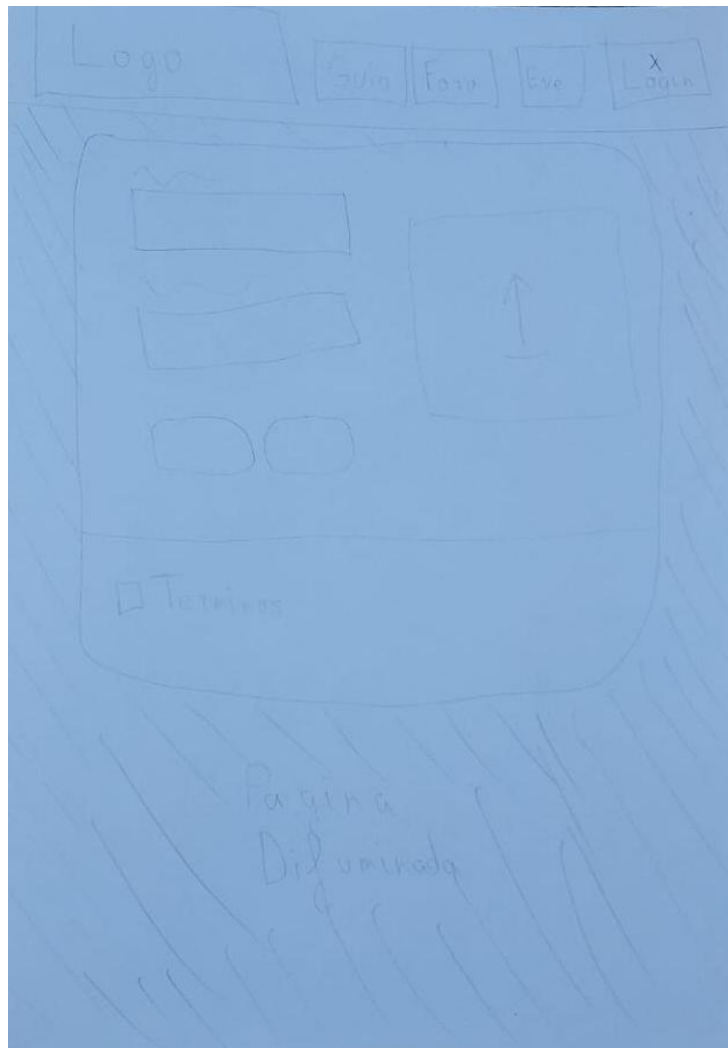
★		1920 x 1080 px	Fondo de la página web
★		55 x 55 px	Cursor de la pagina web
		350 x 148 px	Logo pagina web provisional
		439 x 399 px	1º juego caratula y enlace a su guia
		344 x 357 px	Mezcla de las dos versiones del segundo juego que llevara a la guía conjunta.
		600 x 540 px	La caratula que llevara a la guía conjunta de cada versión del segundo juego.
		731 x 402 px	Mezcla de las dos versiones del tercer juego que llevara a la guía conjunta.
		456 x 409 px	La caratula que llevara a la guía conjunta de cada versión del tercer juego.
		960 x 878 px	Tercera versión del tercer juego
★		353 x 223 px	Mezcla de las dos versiones del cuarto juego que llevara a la guía conjunta.

	510 x 558 px	La caratula que llevara a la guía conjunta de cada versión del cuarto juego.
	521 x 384 px	Mezcla de las dos versiones del quinto juego que llevara a la guía conjunta.
	456 x 459 px	La caratula que llevara a la guía conjunta de cada versión del quinto juego.
	1875 x 800 px	Mezcla de las dos versiones del sexto juego que llevara a la guía conjunta.
	370 x 342 px	La caratula que llevara a la guía conjunta de cada versión del sexto juego.
	380 x 350 px	Caratula del remake de los tres primeros juegos que llevara a su guía. Exclusivo Japón.
	496 x 700 px	Caratula del primer juego para wii que llevara a su guía.
	287 x 407 px	Caratula del segundo juego para wii que llevara a su guía. Exclusivo japon.

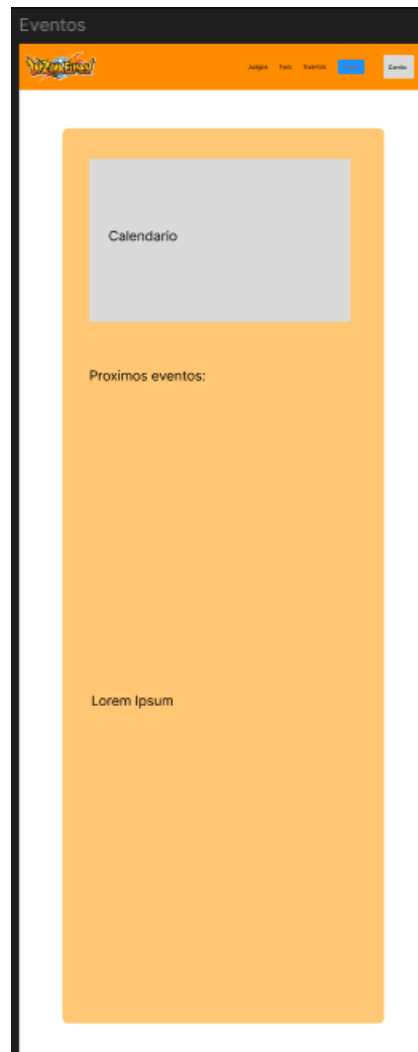
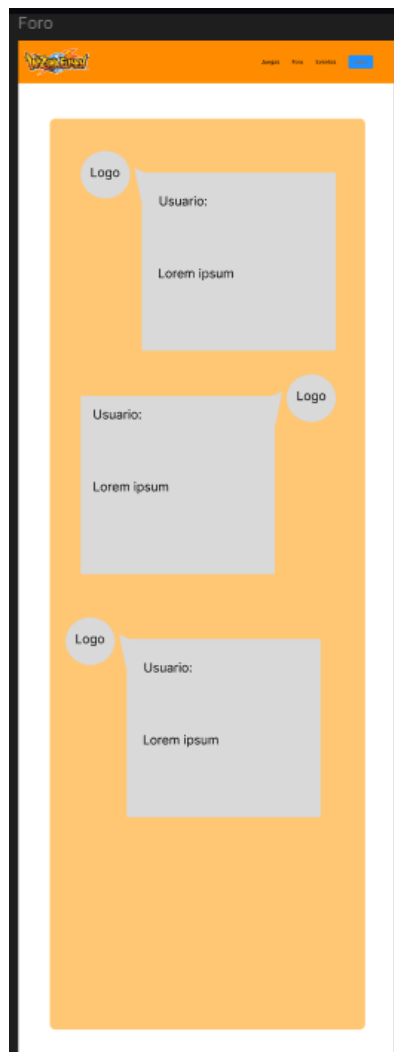
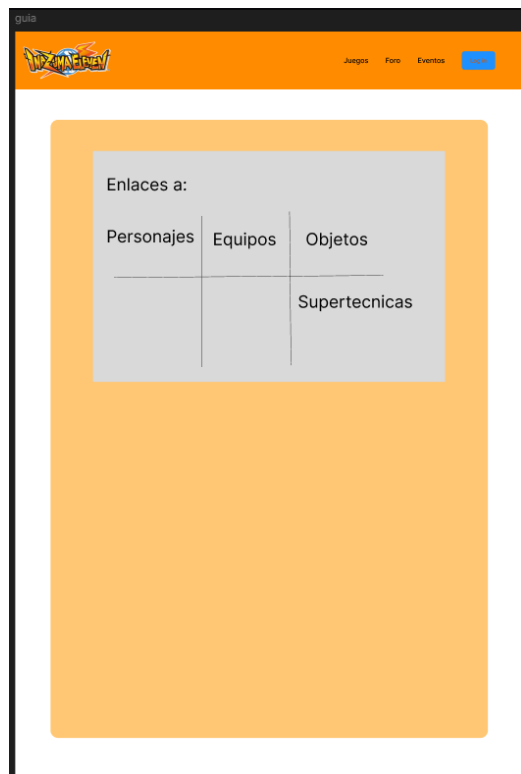
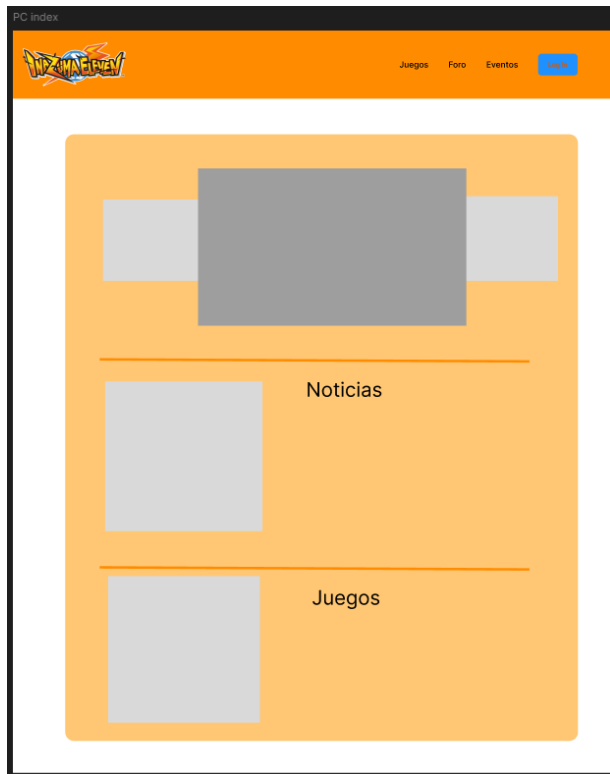
3.2.1.5. Prototipo del sitio web

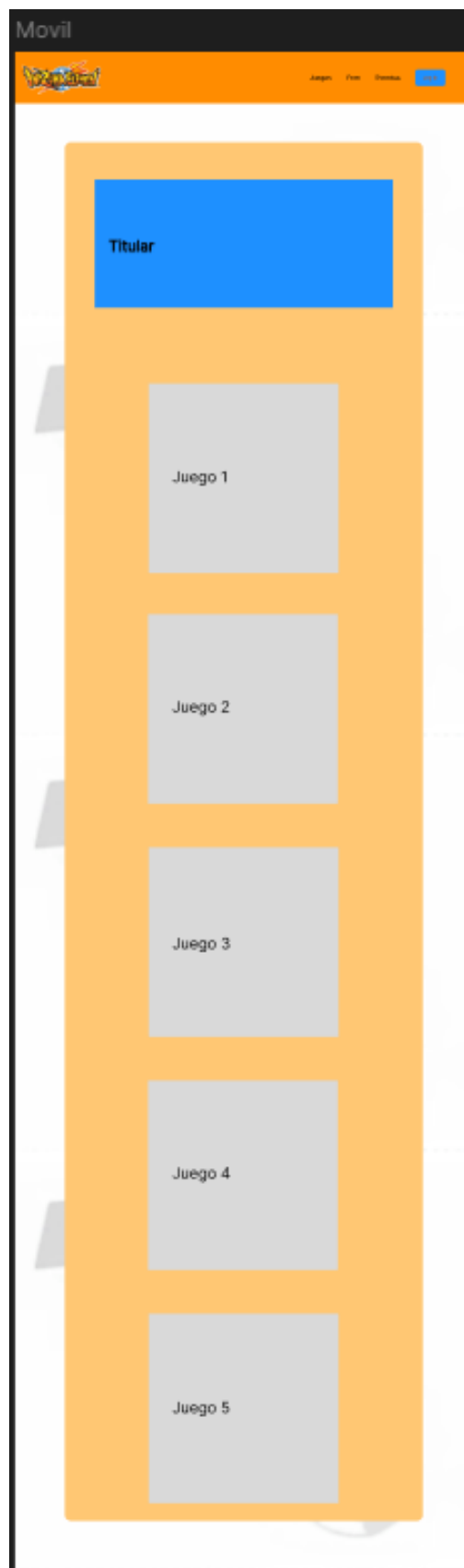
Boceto





Wireframe





Mockup



Noticias:

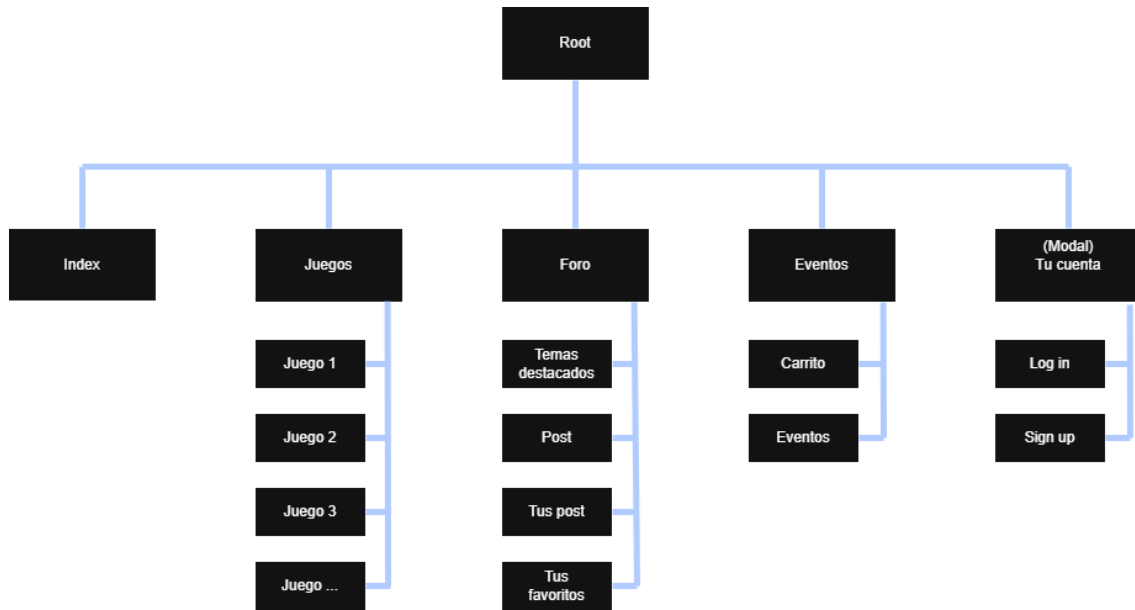


Level-5 ha encendido la cuenta regresiva final para el lanzamiento de Inazuma Eleven: Victory Road, una de las entregas más esperadas por los fans del fútbol y los videojuegos de rol. A tan solo 31 días del estreno oficial, la compañía japonesa ha publicado una nueva imagen promocional protagonizada por Mark Evans (Mamoru Endou).



Inazuma Eleven combina el espíritu del fútbol con la emoción del rol (RPG) en una aventura llena de amistad, esfuerzo y superación.

3.2.1.6. Mapa de navegación



3.2.2. Fase de desarrollo

Para mi fase de desarrollo, he estructurado el trabajo en tres capas clave que me permitirán construir la web de forma ágil y organizada:

- **Capa de Datos:** Es el primer paso. Aquí diseño y monto la base de datos en **MySQL**, estructurando todas las tablas de jugadores, técnicas, equipos y objetos para que las consultas sean instantáneas.
- **Capa Lógica (Backend):** Uso **PHP** para programar las funciones principales: el motor de búsqueda con filtros avanzados, el sistema de registro de usuarios y la lógica detrás del calendario de eventos y torneos.
- **Capa Visual (Frontend):** Con **HTML, CSS (Bootstrap) y JS**, doy vida a la interfaz. Me enfoco en que sea totalmente *responsive* y que respete la estética de Inazuma Eleven, logrando que la navegación sea rápida e inmersiva desde cualquier dispositivo.

3.2.2.1. Base de datos

3.2.2.1.1. Análisis de requisitos de datos de la aplicación

Objetivo

Soportar las funcionalidades principales de INUMANIA: gestión de usuarios, juegos, comunidad, eventos y favoritos/inscripciones a eventos.

a) Identificación de entidades e interrelaciones

Entidades principales:

- **Usuario** - representa a visitantes, registrados y administradores.
 - **Visitante** – puede ver posts, juegos y eventos, pero no interactuar.
 - **Registrado** – puede crear posts, comentar, inscribirse en eventos y marcar favoritos.
 - **Administrador** – puede gestionar usuarios, juegos, eventos y moderar contenido.
- **Post** - publicaciones creadas por usuarios en la comunidad.
- **Comentario** - comentarios asociados a un Post.
- **Evento** - eventos/quedadas/torneos que a los que los usuarios se pueden inscribir.
- **Juego** - Fichas informativas de videojuegos.
- **Equipo** - Equipo, puede ser el tuyo o el rival y existe pachanga(5vs5) y futbol 11
- **Personaje** - Personajes dentro del videojuego.
- **Supertecnica** - Tecnica especial de los personajes
- **Coleccionable** - Objetos dentro del juego se pueden usar para fichar jugadores.

Relaciones importantes:

- Un **Usuario** puede crear muchos **Posts** (1:M).
- Un **Post** puede tener muchos **Comentarios** (1:M).
- Un **Usuario** puede escribir muchos **Comentarios** (1:M).
- Un **Usuario** puede inscribirse en muchos **Eventos** y un **Evento** puede tener muchos **Usuarios** inscritos (M:M → tabla intermedia **InscripcionEvento**).
- Un **Usuario** puede marcar como favorito muchos **Posts** y un **Post** puede ser favorito de muchos **Usuarios** (M:M → tabla **Favorito**).
 - **InscripcionEvento** - relación entre **Usuario** y **Evento**.
 - **Favorito** - relación que permite que un **Usuario** marque **Posts** como favoritos (implementaremos post destacados en el aside).
- Un **Juego** tiene muchos **Equipos** (1:M).
- Un **Juego** tiene muchos **Personajes** (1:M).
- Un **Juego** tiene muchos **Coleccionables** (1:M).
- Un **Personaje** puede pertenecer a muchos **Equipos** y un **Equipo** tiene muchos **Personajes** (M:M → tabla intermedia **personajeEquipo**).
-

- Un **Personaje** tiene muchas **Supertecnicas** y una **Supertecnica** puede ser utilizada por muchos **Personajes** (M:M → tabla intermedia **tecnicasPersonaje**).
 - **PersonajeEquipo** – relación entre **Equipos** y **Personajes**.
 - **TecnicasPersonaje** – relación entre **Supertecnicas** y **Personaje** (Max 6)
 - Un **Personaje** puede estar relacionado con un **Coleccionable** y un **Coleccionable** puede servir para fichar a varios **Personajes** (1:M).
- Un **Coleccionable** pertenece a un único **Juego** (M:1).

b) Identificación de atributos de cada entidad

Usuario: usuario_id, nombre_usuario, email, contraseña, avatar.

Post: post_id, usuario_id, titulo, contenido, categoría.

Comentario: comentario_id, post_id, usuario_id, contenido.

Evento: evento_id, titulo, descripción, fecha_inscripcion, fecha_inicio, fecha_fin, lugar, plazas, precio.

Juego: juego_id, titulo, plataformas, año_lanzamiento, descripcion, imagen.

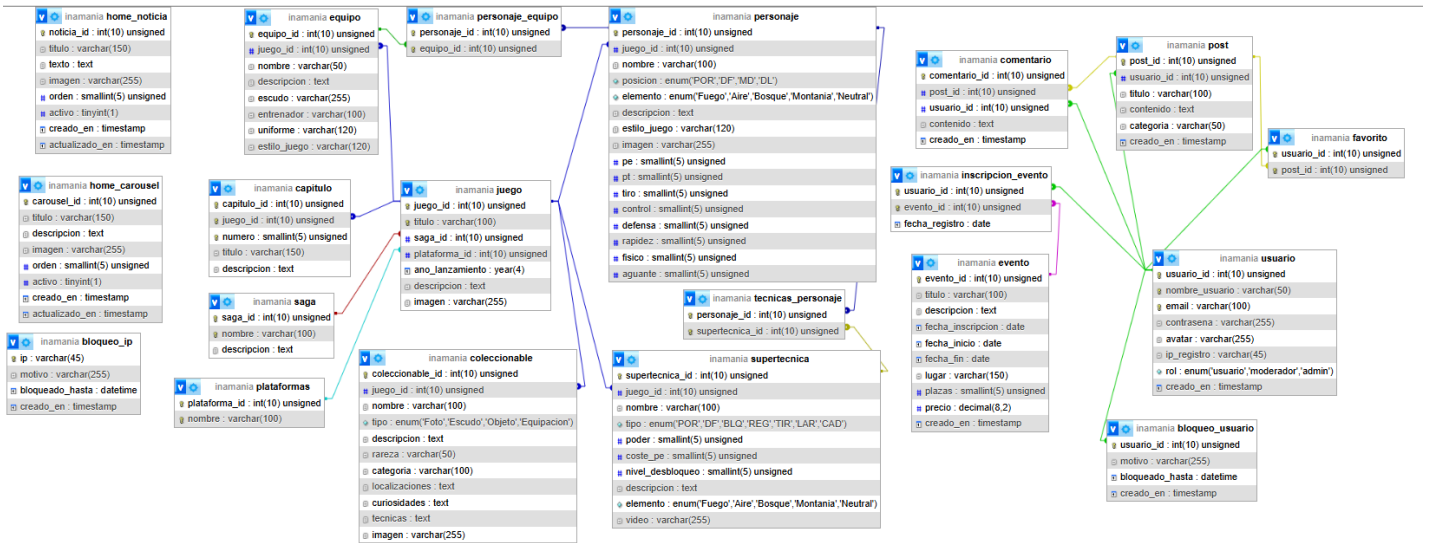
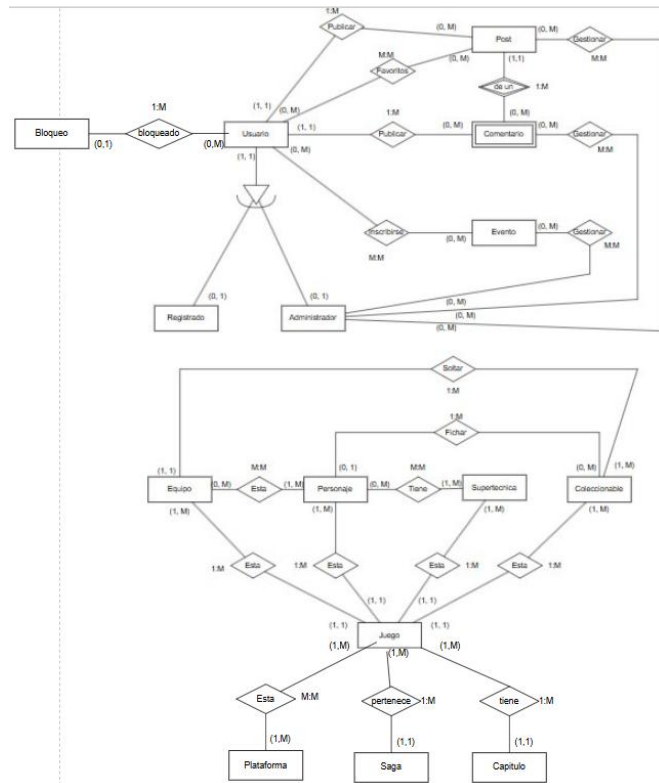
Equipo: equipo_id, juego_id, nombre, descripción, escudo, entrenador.

Personaje: personaje_id, imagen, juego_id, nombre, posición, elemento, descripción.

SuperTécnica: supertecnica_id, juego_id, nombre, tipo, poder, descripcion, elemento, video.

Coleccionable: coleccionable_id, juego_id, nombre, tipo, descripcion, imagen.

3.2.2.1.2 Diseño lógico de datos



3.2.2.1.3 Paso del modelo lógico (E/R) al modelo relacional (tablas)

a. Identificación de atributos de cada tabla, sus tipos y tamaños

Usuario

Atributo	Tipo	Tamaño
usuario_id	INT	10
nombre	VARCHAR2	16
email	VARCHAR2	50
contraseña	VARCHAR2	255
avatar	VARCHAR2	255
ip_registro	VARCHAR2	45
rol	ENUM	"usuario","moderador","admin"
creado_en	TIMESTAMP	-

Post

Atributo	Tipo	Tamaño
post_id	INT	10
usuario_id	INT	10
titulo	VARCHAR2	100
contenido	TEXT	-
categoría	VARCHAR2	50
creado_en	TIMESTAMP	-

Comentario

Atributo	Tipo	Tamaño
comentario_id	INT	10
post_id	INT	10
usuario_id	INT	10
contenido	TEXT	-
creado_en	TIMESTAMP	-

Evento

Atributo	Tipo	Tamaño
evento_id	INT	10
titulo	VARCHAR2	100
descripción	TEXT	-
fecha_inscripcion	DATE	-
fecha_inicio	DATE	-
fecha_fin	DATE	-
lugar	VARCHAR2	150
plazas	INT	6
precio	FLOAT	6,2
creado_en	TIMESTAMP	-

Bloqueo_usuario

Atributo	Tipo	Tamaño
usuario_id	INT	10
motivo	VARCHAR2	255

bloqueado_hasta	DATETIME	-
creado_en	TIMESTAMP	-

Juego

Atributo	Tipo	Tamaño
juego_id	INT	10
titulo	VARCHAR2	100
plataformas	INT	10
saga_id	INT	10
año_lanzamiento	YEAR	-
Descripción	TEXT	-
Imagen	VARCHAR2	255

Equipo

Atributo	Tipo	Tamaño
equipo_id	INT	10
juego_id	INT	10
Nombre	VARCHAR2	50
Descripción	TEXT	-
Escudo	VARCHAR2	255
Entrenador	VARCHAR2	255

Personaje

Atributo	Tipo	Tamaño
personaje_id	INT	10
juego_id	INT	10
Nombre	VARCHAR2	100
Posición	ENUM	-
Elemento	ENUM	-
Descripción	TEXT	-
Imagen	VARCHAR2	255
Cada stat	SMALLINT	5

Supertecnica

Atributo	Tipo	Tamaño
supertecnica_id	INT	10
juego_id	INT	10
Nombre	VARCHAR2	100
Tipo	ENUM	-
Poder	INT	5
Descripción	TEXT	-
Elemento	ENUM	-
Video	VARCHAR2	255

Coleccionable

Atributo	Tipo	Tamaño
coleccionable_id	INT	10
juego_id	INT	10

Nombre	VARCHAR2	100
Tipo	ENUM	-
Descripción	TEXT	-
Imagen	VARCHAR2	255

Saga

Atributo	Tipo	Tamaño
saga_id	INT	10
nombre	VARCHAR2	100
descripción	TEXT	-

Plataformas

Atributo	Tipo	Tamaño
plataforma_id	INT	10
nombre	VARCHAR2	100

Bloqueo_ip

Atributo	Tipo	Tamaño
ip	VARCHAR2	45
motivo	VARCHAR2	255
bloqueado_hasta	DATETIME	-
creado_en	TIMESTAMP	-

Home_carrousel

Atributo	Tipo	Tamaño
carrousel_id	INT	10
titulo	VARCHAR2	150
descripción	TEXT	-
imagen	VARCHAR2	255
orden	SMALLINT	5
activo	TINYINT	1
creado_en	TIMESTAMP	-
actualizado_en	TIMESTAMP	-

Home_noticia

Atributo	Tipo	Tamaño
noticia_id	INT	10
titulo	VARCHAR2	150
texto	TEXT	-
imagen	VARCHAR2	255
orden	SMALLINT	5
activo	TINYINT	1
creado_en	TIMESTAMP	-
actualizado_en	TIMESTAMP	-

Tablas intermedias (relaciones M:M)**Favorito**

Atributo	Tipo	Tamaño
usuario_id	INT	10
post_id	INT	10

InscripcionEvento

Atributo	Tipo	Tamaño
usuario_id	INT	10
evento_id	INT	10
fecha_registro	DATE	

PersonajeEquipo

Atributo	Tipo	Tamaño
personaje_id	INT	10
equipo_id	INT	10

TecnicasPersonaje

Atributo	Tipo	Tamaño
personaje_id	INT	10
supertecnica_id	INT	10

a. Identificación de claves principales y claves candidatas

Claves principales (PK) y candidatas (FK)

- **Usuario:** PK (usuario_id). *Candidatas:* email, nombre_usuario.
- **Post:** PK (post_id). **FK** (usuario_id) → Usuario.
- **Comentario:** PK (comentario_id). **FK** (post_id) → Post, **FK** (usuario_id) → Usuario.
- **Evento:** PK (evento_id).
- **InscripcionEvento:** PK (usuario_id, evento_id). **FK** (usuario_id) → Usuario, **FK** (evento_id) → Evento.
- **Favorito:** PK (usuario_id, post_id). **FK** (usuario_id) → Usuario, **FK** (post_id) → Post.
- **Saga:** PK (saga_id).
- **Juego:** PK (juego_id). **FK** (saga_id) → Saga.
- **Equipo:** PK (equipo_id). **FK** (juego_id) → Juego.
- **Personaje:** PK (personaje_id). **FK** (juego_id) → Juego.
- **Supertecnica:** PK (supertecnica_id). **FK** (juego_id) → Juego.
- **Coleccionable:** PK (coleccionable_id). **FK** (juego_id) → Juego.
- **PersonajeEquipo:** PK (personaje_id, equipo_id). **FK** (personaje_id) → Personaje, **FK** (equipo_id) → Equipo.
- **TecnicasPersonaje:** PK (personaje_id, supertecnica_id). **FK** (personaje_id) → Personaje, **FK** (supertecnica_id) → Supertecnica.

b. Identificación de las reglas de integridad y seguridad de la base de datos

1. **Integridad de Entidad:** Ninguna Clave Primaria (PK) puede ser nula (NOT NULL).
2. **Integridad Referencial:**
 - Si se borra un **Usuario**, se deben borrar sus **Comentarios** y **Favoritos** (ON DELETE CASCADE).
 - No se puede borrar un **Juego** si tiene **Personajes** asociados (protección de datos).
3. **Reglas de Dominio (Restricciones):**
 - email: Debe ser único y tener formato de correo.
 - precio: Debe ser \$\ge 0\$.
 - plazas: Debe ser un número entero positivo.
 - rol: Solo puede ser "usuario", "moderador" o "admin".
4. **Seguridad:**
 - La contraseña debe estar encriptada (Hash).
 - ip_registro se guarda para auditoría y posibles bloqueos.

c. Modelo relacional (tablas)

Módulo Social

- **USUARIO:** usuario_id (PK), nombre_usuario (Unique), email (Unique), contraseña, rol, avatar, ip_registro, creado_en.
- **POST:** post_id (PK), usuario_id (FK), titulo, contenido, categoria, creado_en.
- **COMENTARIO:** comentario_id (PK), post_id (FK), usuario_id (FK), contenido, creado_en.
- **FAVORITO:** usuario_id (PK/FK), post_id (PK/FK).

Módulo de Eventos

- **EVENTO:** evento_id (PK), titulo, descripcion, fecha_inicio, fecha_fin, lugar, plazas, precio.
- **INSCRIPCION_EVENTO:** usuario_id (PK/FK), evento_id (PK/FK), fecha_registro.

Módulo de Información (Wiki)

- **SAGA:** saga_id (PK), nombre, descripcion.
- **JUEGO:** juego_id (PK), saga_id (FK), titulo, año_lanzamiento, descripcion, imagen.
- **EQUIPO:** equipo_id (PK), juego_id (FK), nombre, descripcion, escudo, entrenador.
- **PERSONAJE:** personaje_id (PK), juego_id (FK), nombre, posicion, elemento, descripcion, imagen, stats....
- **SUPERTECNICA:** supertecnica_id (PK), juego_id (FK), nombre, tipo, poder, elemento, descripcion, video.
- **COLECCIONABLE:** coleccionable_id (PK), juego_id (FK), nombre, tipo, descripcion, imagen.
- **PERSONAJE_EQUIPO:** personaje_id (PK/FK), equipo_id (PK/FK).
- **TECNICA_PERSONAJE:** personaje_id (PK/FK), supertecnica_id (PK/FK).

3.2.2.1.4 Aplicación de reglas de normalización al modelo relacional

1º Forma

Se cumple la 1º forma de la normalización cuando creemos una tabla para el atributo plataformas de la entidad Juego, haciendo así que todos los campos sean atómicos y tengan claves principales

Plataforma

- plataforma_id
- nombre

2º Forma

Se cumple desde un inicio la 2º forma de la normalización ya que todos los campos de las entidades dependen directamente de la clave principal, aunque sea compuesta.

3º Forma

Se cumple la 3º forma de la normalización ya que ningún campo depende de un atributo que no sea la clave principal.

3.2.2.1.5 Tipos de datos para el sistema gestor seleccionado

Los tipos de datos son:

- **INT** → Identificadores principales.
- **VARCHAR2(n)** → Textos cortos como nombres o títulos
- **TEXT** → Contenidos o descripciones largas.
- **DATE / YEAR** → Fechas o años.
- **FLOAT(n,m)** → Precios o cantidades monetarias.
- **ENUM** → Valores controlados (posición, elemento, tipo de técnica).
- **VARCHAR2(255)** → Rutas a imágenes, vídeos o contraseñas.

Ejemplos específicos:

- contraseña → VARCHAR2(255) para almacenar hash seguro.
- elemento → ENUM('Fuego', 'Aire', 'Bosque', 'Montaña', 'Neutral').
- Precio → FLOAT(6,2) para poder tener un buen rango de precios en euros con centimos.

3.2.2.1.6 Scripts de creación de tablas e inserciones iniciales

```
SET NAMES utf8mb4;
```

```
CREATE DATABASE IF NOT EXISTS `inamania`  
  CHARACTER SET utf8mb4  
  COLLATE utf8mb4_unicode_ci;
```

```
USE `inamania`;
```

```
SET FOREIGN_KEY_CHECKS = 0;
```

```
DROP TABLE IF EXISTS tecnicas_personaje;  
DROP TABLE IF EXISTS personaje_equipo;  
DROP TABLE IF EXISTS coleccionable;  
DROP TABLE IF EXISTS supertecnica;  
DROP TABLE IF EXISTS personaje;  
DROP TABLE IF EXISTS equipo;  
DROP TABLE IF EXISTS capitulo;  
DROP TABLE IF EXISTS favorito;  
DROP TABLE IF EXISTS inscripcion_evento;  
DROP TABLE IF EXISTS bloqueo_ip;  
DROP TABLE IF EXISTS bloqueo_usuario;  
DROP TABLE IF EXISTS home_noticia;  
DROP TABLE IF EXISTS home_carousel;  
DROP TABLE IF EXISTS comentario;  
DROP TABLE IF EXISTS post;  
DROP TABLE IF EXISTS evento;  
DROP TABLE IF EXISTS juego;  
DROP TABLE IF EXISTS saga;  
DROP TABLE IF EXISTS plataformas;  
DROP TABLE IF EXISTS usuario;
```

```
SET FOREIGN_KEY_CHECKS = 1;
```

```
CREATE TABLE usuario (  
  usuario_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
```



```
nombre_usuario VARCHAR(50) NOT NULL UNIQUE,  
email VARCHAR(100) NOT NULL UNIQUE,  
contrasena VARCHAR(255) NOT NULL,  
avatar VARCHAR(255) DEFAULT NULL,  
ip_registro VARCHAR(45) DEFAULT NULL,  
rol ENUM('usuario', 'moderador', 'admin') NOT NULL DEFAULT  
'usuario',  
creado_en TIMESTAMP NOT NULL DEFAULT  
CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

```
CREATE TABLE bloqueo_usuario (  
    usuario_id INT UNSIGNED PRIMARY KEY,  
    motivo VARCHAR(255) DEFAULT NULL,  
    bloqueado_hasta DATETIME DEFAULT NULL,  
    creado_en TIMESTAMP NOT NULL DEFAULT  
CURRENT_TIMESTAMP,  
    CONSTRAINT fk_bloqueo_usuario_usuario  
        FOREIGN KEY (usuario_id)  
        REFERENCES usuario(usuario_id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

```
CREATE TABLE bloqueo_ip (  
    ip VARCHAR(45) PRIMARY KEY,  
    motivo VARCHAR(255) DEFAULT NULL,  
    bloqueado_hasta DATETIME DEFAULT NULL,  
    creado_en TIMESTAMP NOT NULL DEFAULT  
CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

```
CREATE TABLE home_carousel (  
    carousel_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    titulo VARCHAR(150) NOT NULL,  
    descripcion TEXT,  
    imagen VARCHAR(255) NOT NULL,  
    orden SMALLINT UNSIGNED NOT NULL DEFAULT 1,
```

```
    activo TINYINT(1) NOT NULL DEFAULT 1,  
    creado_en TIMESTAMP NOT NULL DEFAULT  
CURRENT_TIMESTAMP,  
    actualizado_en TIMESTAMP NOT NULL DEFAULT  
CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

```
CREATE TABLE home_noticia (  
    noticia_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    titulo VARCHAR(150) NOT NULL,  
    texto TEXT NOT NULL,  
    imagen VARCHAR(255) NOT NULL,  
    orden SMALLINT UNSIGNED NOT NULL DEFAULT 1,  
    activo TINYINT(1) NOT NULL DEFAULT 1,  
    creado_en TIMESTAMP NOT NULL DEFAULT  
CURRENT_TIMESTAMP,  
    actualizado_en TIMESTAMP NOT NULL DEFAULT  
CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

```
CREATE TABLE plataformas (  
    plataforma_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL UNIQUE  
) ENGINE=InnoDB;
```

```
CREATE TABLE saga (  
    saga_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL UNIQUE,  
    descripcion TEXT  
) ENGINE=InnoDB;
```

```
CREATE TABLE juego (  
    juego_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    titulo VARCHAR(100) NOT NULL UNIQUE,  
    saga_id INT UNSIGNED DEFAULT NULL,  
    plataforma_id INT UNSIGNED DEFAULT NULL,  
    ano_lanzamiento YEAR DEFAULT NULL,  
    descripcion TEXT,
```

```
imagen VARCHAR(255),  
CONSTRAINT fk_juego_saga  
    FOREIGN KEY (saga_id)  
    REFERENCES saga(saga_id)  
    ON DELETE SET NULL  
    ON UPDATE CASCADE,  
CONSTRAINT fk_juego_plataforma  
    FOREIGN KEY (plataforma_id)  
    REFERENCES plataformas(plataforma_id)  
    ON DELETE SET NULL  
    ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

```
CREATE TABLE post (  
    post_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    usuario_id INT UNSIGNED NOT NULL,  
    titulo VARCHAR(100) NOT NULL,  
    contenido TEXT NOT NULL,  
    categoria VARCHAR(50),  
    creado_en TIMESTAMP NOT NULL DEFAULT  
CURRENT_TIMESTAMP,  
    CONSTRAINT fk_post_usuario  
        FOREIGN KEY (usuario_id)  
        REFERENCES usuario(usuario_id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

```
CREATE TABLE comentario (  
    comentario_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    post_id INT UNSIGNED NOT NULL,  
    usuario_id INT UNSIGNED NOT NULL,  
    contenido TEXT NOT NULL,  
    creado_en TIMESTAMP NOT NULL DEFAULT  
CURRENT_TIMESTAMP,  
    CONSTRAINT fk_comentario_post  
        FOREIGN KEY (post_id)  
        REFERENCES post(post_id)
```

```
    ON DELETE CASCADE
    ON UPDATE CASCADE,
CONSTRAINT fk_comentario_usuario
    FOREIGN KEY (usuario_id)
    REFERENCES usuario(usuario_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE
) ENGINE=InnoDB;
```

```
CREATE TABLE evento (
    evento_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    titulo VARCHAR(100) NOT NULL,
    descripcion TEXT,
    fecha_inscripcion DATE,
    fecha_inicio DATE,
    fecha_fin DATE,
    lugar VARCHAR(150),
    plazas SMALLINT UNSIGNED,
    precio DECIMAL(8,2) NOT NULL DEFAULT 0.00,
    creado_en TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

```
CREATE TABLE inscripcion_evento (
    usuario_id INT UNSIGNED NOT NULL,
    evento_id INT UNSIGNED NOT NULL,
    fecha_registro DATE NOT NULL DEFAULT (CURRENT_DATE),
    PRIMARY KEY (usuario_id, evento_id),
    CONSTRAINT fk_inscripcion_usuario
        FOREIGN KEY (usuario_id)
        REFERENCES usuario(usuario_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_inscripcion_evento
        FOREIGN KEY (evento_id)
        REFERENCES evento(evento_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
```

```
) ENGINE=InnoDB;
```

```
CREATE TABLE favorito (  
    usuario_id INT UNSIGNED NOT NULL,  
    post_id INT UNSIGNED NOT NULL,  
    PRIMARY KEY (usuario_id, post_id),  
    CONSTRAINT fk_favorito_usuario  
        FOREIGN KEY (usuario_id)  
        REFERENCES usuario(usuario_id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE,  
    CONSTRAINT fk_favorito_post  
        FOREIGN KEY (post_id)  
        REFERENCES post(post_id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

```
CREATE TABLE equipo (  
    equipo_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    juego_id INT UNSIGNED NOT NULL,  
    nombre VARCHAR(50) NOT NULL,  
    descripcion TEXT,  
    escudo VARCHAR(255),  
    entrenador VARCHAR(100),  
    uniforme VARCHAR(120),  
    estilo_juego VARCHAR(120),  
    CONSTRAINT fk_equipo_juego  
        FOREIGN KEY (juego_id)  
        REFERENCES juego(juego_id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

```
CREATE TABLE personaje (  
    personaje_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    juego_id INT UNSIGNED NOT NULL,
```

```
nombre VARCHAR(100) NOT NULL,  
posicion ENUM('POR', 'DF', 'MD', 'DL') NOT NULL,  
elemento ENUM('Fuego', 'Aire', 'Bosque', 'Montania', 'Neutral') NOT  
NULL,  
descripcion TEXT,  
estilo_juego VARCHAR(120),  
imagen VARCHAR(255),  
pe SMALLINT UNSIGNED DEFAULT NULL,  
pt SMALLINT UNSIGNED DEFAULT NULL,  
tiro SMALLINT UNSIGNED DEFAULT NULL,  
control SMALLINT UNSIGNED DEFAULT NULL,  
defensa SMALLINT UNSIGNED DEFAULT NULL,  
rapidez SMALLINT UNSIGNED DEFAULT NULL,  
fisico SMALLINT UNSIGNED DEFAULT NULL,  
aguante SMALLINT UNSIGNED DEFAULT NULL,  
CONSTRAINT fk_personaje_juego  
    FOREIGN KEY (juego_id)  
    REFERENCES juego(juego_id)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

```
CREATE TABLE supertecnica (  
    supertecnica_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    juego_id INT UNSIGNED NOT NULL,  
    nombre VARCHAR(100) NOT NULL,  
    tipo ENUM('POR', 'DF', 'BLQ', 'REG', 'TIR', 'LAR', 'CAD') NOT NULL,  
    poder SMALLINT UNSIGNED,  
    coste_pe SMALLINT UNSIGNED,  
    nivel_desbloqueo SMALLINT UNSIGNED,  
    descripcion TEXT,  
    elemento ENUM('Fuego', 'Aire', 'Bosque', 'Montania', 'Neutral') NOT  
NULL,  
    video VARCHAR(255),  
    CONSTRAINT fk_supertecnica_juego  
        FOREIGN KEY (juego_id)  
        REFERENCES juego(juego_id)  
        ON DELETE CASCADE
```

```
        ON UPDATE CASCADE
    ) ENGINE=InnoDB;

CREATE TABLE coleccionable (
    coleccionable_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    juego_id INT UNSIGNED NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    tipo ENUM('Foto', 'Escudo', 'Objeto', 'Equipacion') NOT NULL,
    descripcion TEXT,
    rareza VARCHAR(50) DEFAULT NULL,
    categoria VARCHAR(100) DEFAULT NULL,
    localizaciones TEXT,
    curiosidades TEXT,
    tecnicas TEXT,
    imagen VARCHAR(255),
    CONSTRAINT fk_coleccionable_juego
        FOREIGN KEY (juego_id)
        REFERENCES juego(juego_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
    ) ENGINE=InnoDB;
```

```
CREATE TABLE capitulo (
    capitulo_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    juego_id INT UNSIGNED NOT NULL,
    numero SMALLINT UNSIGNED NOT NULL,
    titulo VARCHAR(150) NOT NULL,
    descripcion TEXT,
    CONSTRAINT fk_capitulo_juego
        FOREIGN KEY (juego_id)
        REFERENCES juego(juego_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    UNIQUE KEY uq_capitulo_juego_numero (juego_id, numero)
    ) ENGINE=InnoDB;
```

```
CREATE TABLE personaje_equipo (
```

```
personaje_id INT UNSIGNED NOT NULL,  
equipo_id INT UNSIGNED NOT NULL,  
PRIMARY KEY (personaje_id, equipo_id),  
CONSTRAINT fk_personaje_equipo_personaje  
    FOREIGN KEY (personaje_id)  
    REFERENCES personaje(personaje_id)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE,  
CONSTRAINT fk_personaje_equipo_equipo  
    FOREIGN KEY (equipo_id)  
    REFERENCES equipo(equipo_id)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

```
CREATE TABLE tecnicas_personaje (  
    personaje_id INT UNSIGNED NOT NULL,  
    supertecnica_id INT UNSIGNED NOT NULL,  
    PRIMARY KEY (personaje_id, supertecnica_id),  
    CONSTRAINT fk_tecnicas_personaje_personaje  
        FOREIGN KEY (personaje_id)  
        REFERENCES personaje(personaje_id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE,  
    CONSTRAINT fk_tecnicas_personaje_supertecnica  
        FOREIGN KEY (supertecnica_id)  
        REFERENCES supertecnica(supertecnica_id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
) ENGINE=InnoDB;
```


3.2.2.2. Servidor

3.2.2.2.1. Lista de funciones en php

Archivo	Función	Descripción
bootstrap.php	route	Construye la URL interna para una página.
bootstrap.php	asset	Construye la URL de un recurso estatico.
AdminController.php	uploadImage	Devuelve un texto o ruta procesada.
AdminController.php	admin	Controla el flujo principal de la acción o página.
CarritoController.php	isAjaxRequest	Comprueba una condición y devuelve verdadero o falso.
CarritoController.php	todayDate	Obtiene y devuelve información calculada o almacenada.
CarritoController.php	validateAndEnrollUserInEvent	Valida o transforma datos de entrada.
CarritoController.php	validateAndUnenrollUserFromEvent	Valida o transforma datos de entrada.
CarritoController.php	getCarrito	Obtiene y devuelve información calculada o almacenada.
CarritoController.php	setCarrito	Guarda o actualiza un valor interno.
CarritoController.php	addToCart	Crea un registro o realiza un alta.
CarritoController.php	viewCart	Controla el flujo principal de la acción o página.
CarritoController.php	removeFromCart	Elimina, bloquea o desbloquea registros según acción.
CarritoController.php	joinFreeEvent	Crea un registro o realiza un alta.
CarritoController.php	leaveEvent	Elimina, bloquea o desbloquea registros según acción.
CarritoController.php	checkout	Controla el flujo principal de la acción o página.
ForoController.php	foroStyles	Devuelve estilos o recursos de la vista.
ForoController.php	validatedSessionUserId	Valida o transforma datos de entrada.
ForoController.php	currentForumView	Obtiene y devuelve información calculada o almacenada.
ForoController.php	foroRouteWithView	Devuelve un texto o ruta procesada.
ForoController.php	createPostIfNeeded	Crea un registro o realiza un alta.
ForoController.php	foro	Controla el flujo principal de la acción o página.
JuegoController.php	__construct	Inicializa la clase y sus dependencias.
JuegoController.php	juegos	Controla el flujo principal de la acción o página.
JuegoController.php	game	Controla el flujo principal de la acción o página.
JuegoController.php	renderCurrentWiki	Renderiza una vista con datos.
JuegoController.php	renderWiki	Renderiza una vista con datos.
LoginController.php	homeStyles	Devuelve estilos o recursos de la vista.
LoginController.php	authViewData	Devuelve datos estructurados en forma de arreglo.
LoginController.php	handleLoginRequest	Devuelve datos estructurados en forma de arreglo.
LoginController.php	handleLogoutRequest	Devuelve datos estructurados en forma de arreglo.
LoginController.php	handleRegisterRequest	Devuelve datos estructurados en forma de arreglo.
LoginController.php	login	Controla el flujo principal de la acción o página.
ModeratorController.php	uploadImage	Devuelve un texto o ruta procesada.
ModeratorController.php	parseDateOrNull	Valida o transforma datos de entrada.
ModeratorController.php	parseDateTimeLocalOrNull	Valida o transforma datos de entrada.
ModeratorController.php	handleModeratorAction	Devuelve datos estructurados en forma de arreglo.
ModeratorController.php	moderacion	Controla el flujo principal de la acción o página.

PageController.php	baseStyles	Devuelve estilos o recursos de la vista.
PageController.php	homeStyles	Devuelve estilos o recursos de la vista.
PageController.php	renderInfoPage	Renderiza una vista con datos.
PageController.php	home	Ejecuta una acción sin devolver datos.
PageController.php	eventos	Controla el flujo principal de la acción o página.
PageController.php	terminos	Controla el flujo principal de la acción o página.
PageController.php	privacidad	Controla el flujo principal de la acción o página.
PageController.php	avisoLegal	Controla el flujo principal de la acción o página.
PageController.php	notFound	Controla el flujo principal de la acción o página.
PerfilController.php	validateAndStoreAvatar	Valida o transforma datos de entrada.
PerfilController.php	deleteLocalAvatarIfNeeded	Elimina, bloquea o desbloquea registros según acción.
PerfilController.php	renderProfile	Renderiza una vista con datos.
PerfilController.php	perfil	Controla el flujo principal de la acción o página.
Controller.php	sessionUser	Obtiene y devuelve información calculada o almacenada.
Controller.php	currentSessionUserId	Obtiene y devuelve información calculada o almacenada.
Controller.php	currentSessionUserRole	Obtiene y devuelve información calculada o almacenada.
Controller.php	isModeratorOrAdmin	Comprueba una condición y devuelve verdadero o falso.
Controller.php	isAdmin	Comprueba una condición y devuelve verdadero o falso.
Controller.php	consumeFlashSuccess	Lee y limpia un mensaje temporal de sesión.
Controller.php	consumeFlashError	Lee y limpia un mensaje temporal de sesión.
Controller.php	setFlashSuccess	Guarda o actualiza un valor interno.
Controller.php	setFlashError	Guarda o actualiza un valor interno.
Controller.php	render	Renderiza una vista con datos.
View.php	make	Renderiza una vista con datos.
adminModel.php	__construct	Inicializa la clase y sus dependencias.
adminModel.php	listSagas	Lista y devuelve una colección de registros.
adminModel.php	createSaga	Crea un registro o realiza un alta.
adminModel.php	updateSaga	Actualiza datos existentes.
adminModel.php	deleteSaga	Elimina, bloquea o desbloquea registros según acción.
adminModel.php	listPlataformas	Lista y devuelve una colección de registros.
adminModel.php	createPlataforma	Crea un registro o realiza un alta.
adminModel.php	updatePlataforma	Actualiza datos existentes.
adminModel.php	deletePlataforma	Elimina, bloquea o desbloquea registros según acción.
adminModel.php	listJuegos	Lista y devuelve una colección de registros.
adminModel.php	createJuego	Crea un registro o realiza un alta.
adminModel.php	updateJuego	Actualiza datos existentes.
adminModel.php	deleteJuego	Elimina, bloquea o desbloquea registros según acción.
adminModel.php	getJuegoById	Obtiene y devuelve información calculada o almacenada.
adminModel.php	listPersonajes	Lista y devuelve una colección de registros.
adminModel.php	createPersonaje	Crea un registro o realiza un alta.
adminModel.php	updatePersonaje	Actualiza datos existentes.
adminModel.php	deletePersonaje	Elimina, bloquea o desbloquea registros según acción.
adminModel.php	getPersonajeById	Obtiene y devuelve información calculada o

		almacenada.
adminModel.php	listEquipos	Lista y devuelve una colección de registros.
adminModel.php	createEquipo	Crea un registro o realiza un alta.
adminModel.php	updateEquipo	Actualiza datos existentes.
adminModel.php	deleteEquipo	Elimina, bloquea o desbloquea registros según acción.
adminModel.php	getEquipoById	Obtiene y devuelve informacion calculada o almacenada.
conexion.php	conexion	Abre y devuelve la conexión a base de datos.
EventoModel.php	__construct	Inicializa la clase y sus dependencias.
EventoModel.php	listEventsBetween	Lista y devuelve una colección de registros.
EventoModel.php	listRecommendedEventsBy Registrations	Lista y devuelve una colección de registros.
EventoModel.php	listUpcomingEvents	Lista y devuelve una colección de registros.
EventoModel.php	listUpcomingOpenRegistrati onEvents	Lista y devuelve una colección de registros.
EventoModel.php	findEventWithRegistrations ById	Busca un registro y devuelve su resultado.
EventoModel.php	isUserEnrolledInEvent	Comprueba una condición y devuelve verdadero o falso.
EventoModel.php	enrollUserInEvent	Crea un registro o realiza un alta.
EventoModel.php	unenrollUserFromEvent	Devuelve verdadero o falso según el resultado.
EventoModel.php	listEnrolledEventsByUser	Lista y devuelve una colección de registros.
EventoModel.php	createEvent	Crea un registro o realiza un alta.
ForoModel.php	__construct	Inicializa la clase y sus dependencias.
ForoModel.php	listPosts	Lista y devuelve una colección de registros.
ForoModel.php	countPosts	Cuenta registros y devuelve el total.
ForoModel.php	listCommentsByPostIds	Lista y devuelve una colección de registros.
ForoModel.php	listLatestPosts	Lista y devuelve una colección de registros.
ForoModel.php	listCategories	Lista y devuelve una colección de registros.
ForoModel.php	listPopularPosts	Lista y devuelve una colección de registros.
ForoModel.php	createPost	Crea un registro o realiza un alta.
ForoModel.php	postExists	Verifica si un dato ya existe.
ForoModel.php	createComment	Crea un registro o realiza un alta.
ForoModel.php	toggleFavorite	Alterna el estado de un registro.
JuegoModel.php	__construct	Inicializa la clase y sus dependencias.
JuegoModel.php	listGamesGroupedBySaga	Lista y devuelve una colección de registros.
JuegoModel.php	listMenuGames	Lista y devuelve una colección de registros.
JuegoModel.php	getGameWikiByRoute	Obtiene y devuelve información calculada o almacenada.
JuegoModel.php	getGameIdByRouteKey	Obtiene y devuelve información calculada o almacenada.
JuegoModel.php	getChaptersByRouteKey	Obtiene y devuelve información calculada o almacenada.
JuegoModel.php	getTeamsByRouteKey	Obtiene y devuelve información calculada o almacenada.
JuegoModel.php	getCharactersByRouteKey	Obtiene y devuelve información calculada o almacenada.
JuegoModel.php	getSupertecnicasByRouteKey	Obtiene y devuelve información calculada o almacenada.
JuegoModel.php	getObjectsByRouteKey	Obtiene y devuelve información calculada o almacenada.
JuegoModel.php	parseCsvInts	Valida o transforma datos de entrada.
JuegoModel.php	parseCsvStrings	Valida o transforma datos de entrada.

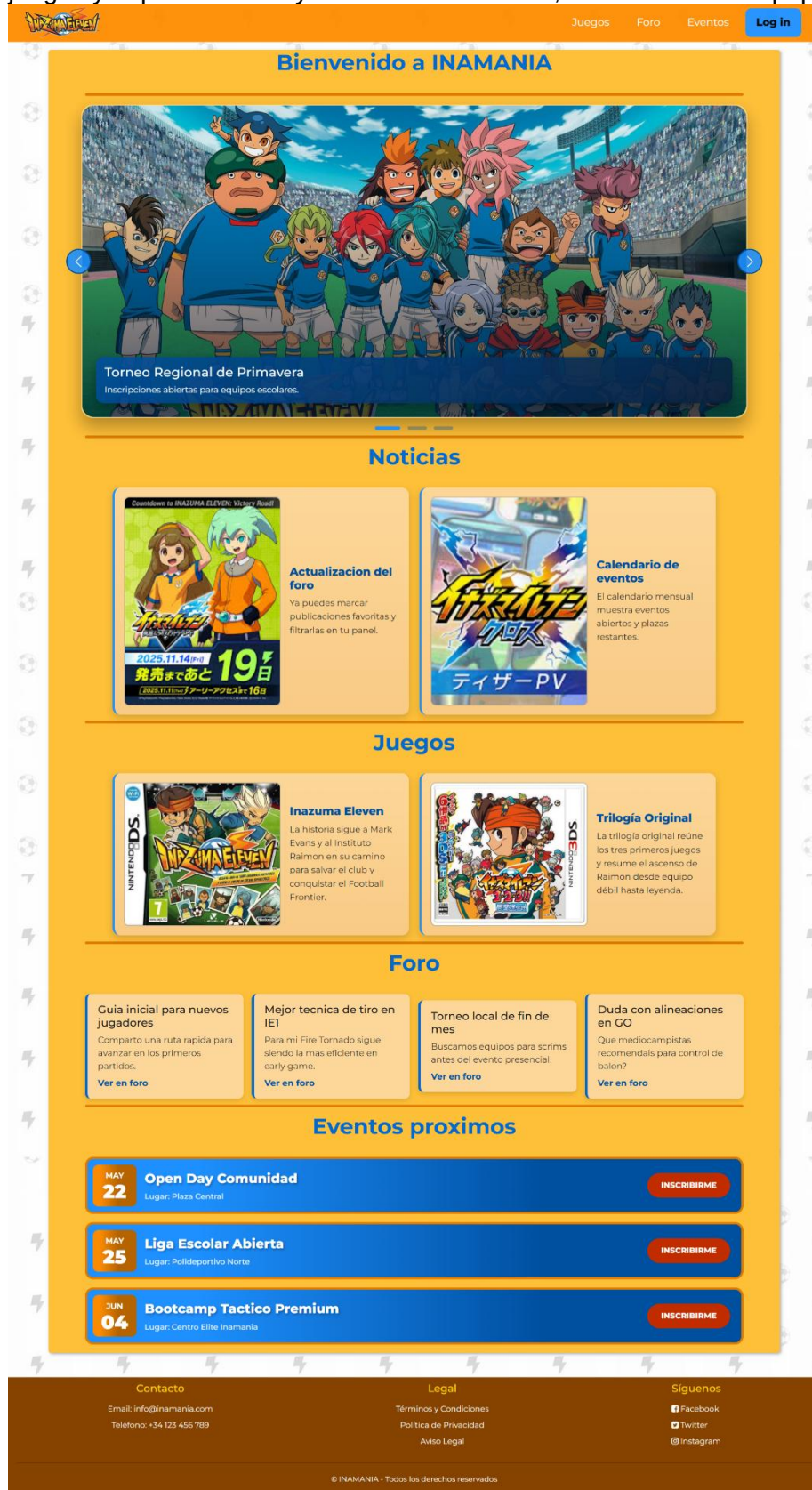
JuegoModel.php	parseIdNamePairs	Valida o transforma datos de entrada.
JuegoModel.php	getObjectDocumentsByGameId	Obtiene y devuelve información calculada o almacenada.
JuegoModel.php	findRelatedObjectIdsByGameIdAndKeywords	Busca un registro y devuelve su resultado.
JuegoModel.php	getTitleByRouteKey	Obtiene y devuelve información calculada o almacenada.
JuegoModel.php	resolveRouteKey	Devuelve un texto o ruta procesada.
JuegoModel.php	ensureRouteMapsLoaded	Garantiza la existencia o consistencia de estructura interna.
JuegoModel.php	resolveSagaPrefix	Devuelve un texto o ruta procesada.
JuegoModel.php	createChapter	Crea un registro o realiza un alta.
JuegoModel.php	updateChapter	Actualiza datos existentes.
JuegoModel.php	createTeam	Crea un registro o realiza un alta.
JuegoModel.php	updateTeam	Actualiza datos existentes.
JuegoModel.php	syncCharacterTeam	Actualiza datos existentes.
JuegoModel.php	createCharacter	Crea un registro o realiza un alta.
JuegoModel.php	updateCharacter	Actualiza datos existentes.
JuegoModel.php	createSupertecnica	Crea un registro o realiza un alta.
JuegoModel.php	updateSupertecnica	Actualiza datos existentes.
JuegoModel.php	createObject	Crea un registro o realiza un alta.
JuegoModel.php	updateObject	Actualiza datos existentes.
ModeratorModel.php	construct	Inicializa la clase y sus dependencias.
ModeratorModel.php	ensureModeratorTables	Garantiza la existencia o consistencia de estructura interna.
ModeratorModel.php	listUsersWithStatus	Lista y devuelve una colección de registros.
ModeratorModel.php	blockUser	Elimina, bloquea o desbloquea registros según acción.
ModeratorModel.php	unblockUser	Elimina, bloquea o desbloquea registros según acción.
ModeratorModel.php	updateUserRole	Actualiza datos existentes.
ModeratorModel.php	listIpBlocks	Lista y devuelve una colección de registros.
ModeratorModel.php	blockIp	Elimina, bloquea o desbloquea registros según acción.
ModeratorModel.php	unblockIp	Elimina, bloquea o desbloquea registros según acción.
ModeratorModel.php	listPostsForModeration	Lista y devuelve una colección de registros.
ModeratorModel.php	deletePost	Elimina, bloquea o desbloquea registros según acción.
ModeratorModel.php	listCommentsForModeration	Lista y devuelve una colección de registros.
ModeratorModel.php	deleteComment	Elimina, bloquea o desbloquea registros según acción.
ModeratorModel.php	listEventsForModeration	Lista y devuelve una colección de registros.
ModeratorModel.php	createEvent	Crea un registro o realiza un alta.
ModeratorModel.php	updateEvent	Actualiza datos existentes.
ModeratorModel.php	listCarouselItems	Lista y devuelve una colección de registros.
ModeratorModel.php	listActiveCarouselItems	Lista y devuelve una colección de registros.
ModeratorModel.php	createCarouselItem	Crea un registro o realiza un alta.
ModeratorModel.php	updateCarouselItem	Actualiza datos existentes.
ModeratorModel.php	listNewsItems	Lista y devuelve una colección de registros.
ModeratorModel.php	listActiveNewsItems	Lista y devuelve una colección de registros.
ModeratorModel.php	createNewsItem	Crea un registro o realiza un alta.
ModeratorModel.php	updateNewsItem	Actualiza datos existentes.

ModeratorModel.php	getCarousellItemById	Obtiene y devuelve información calculada o almacenada.
ModeratorModel.php	getNewsItemById	Obtiene y devuelve información calculada o almacenada.
ModeratorModel.php	deleteEvent	Elimina, bloquea o desbloquea registros según acción.
ModeratorModel.php	deleteCarousellItem	Elimina, bloquea o desbloquea registros según acción.
ModeratorModel.php	deleteNewsItem	Elimina, bloquea o desbloquea registros según acción.
PerfilModel.php	__construct	Inicializa la clase y sus dependencias.
PerfilModel.php	findById	Busca un registro y devuelve su resultado.
PerfilModel.php	usernameExistsForOther	Verifica si un dato ya existe.
PerfilModel.php	updateProfile	Actualiza datos existentes.
userModel.php	__construct	Inicializa la clase y sus dependencias.
userModel.php	ensureRegistrationInColumn	Garantiza la existencia o consistencia de estructura interna.
userModel.php	usernameExists	Verifica si un dato ya existe.
userModel.php	emailExists	Verifica si un dato ya existe.
userModel.php	usernameExistsForOther	Verifica si un dato ya existe.
userModel.php	createUser	Crea un registro o realiza un alta.
userModel.php	authenticateByEmail	Autentica al usuario con email y contraseña.
userModel.php	findById	Busca un registro y devuelve su resultado.
userModel.php	getActiveUserBlock	Obtiene y devuelve información calculada o almacenada.
userModel.php	getActiveIpBlock	Obtiene y devuelve información calculada o almacenada.
userModel.php	updateProfile	Actualiza datos existentes.

3.2.2.3. Cliente

3.2.2.3.1. Diseño de la interfaz

En la pagina de inicio veremos unos elementos básicos, el moderador ajustara el carrousel, las noticias y los juegos y la parte de foro y eventos es dinámica, sacando los mas populares.



El usuario no podrá interactuar con el apartado social si no inicia sesión, hay una opción para registrarse y otra para iniciar sesión, los únicos requisitos son que los campos estén rellenos, un correo válido y una contraseña de más de 6 dígitos, si pulsas en el icono del ojo podrás ver la contraseña.

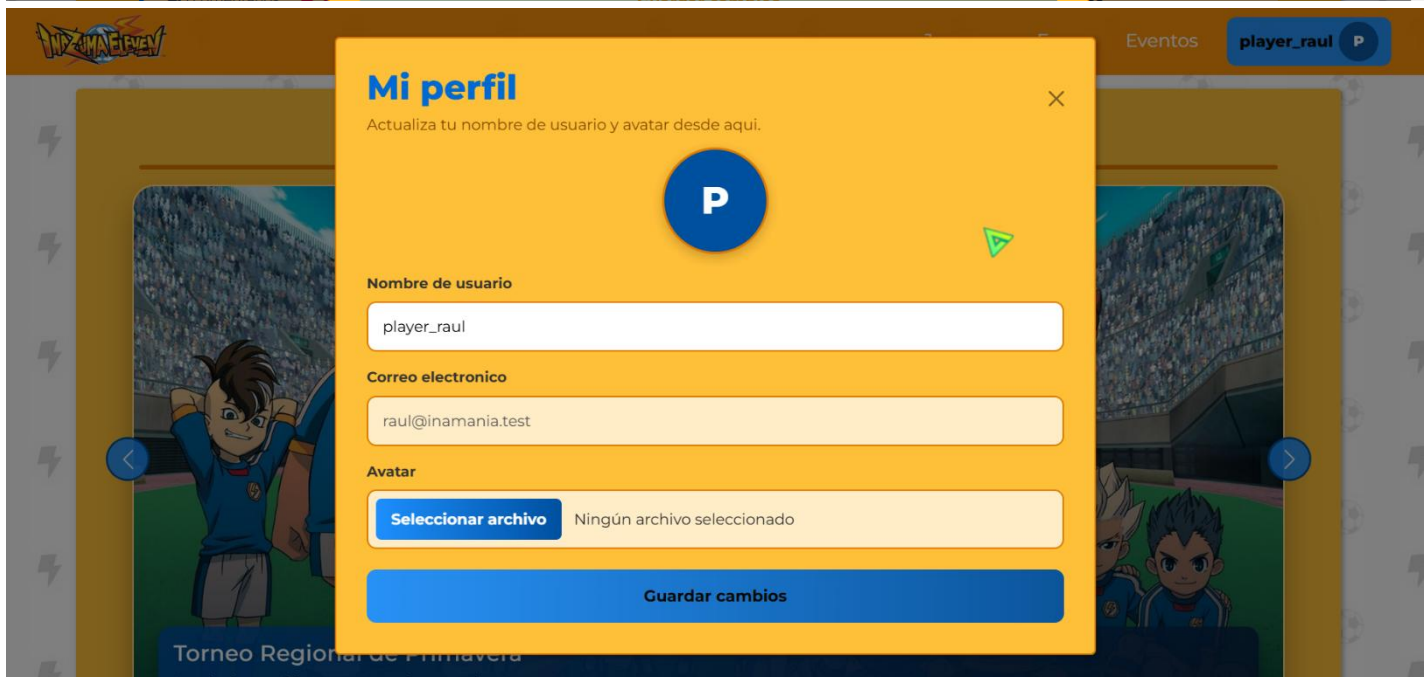
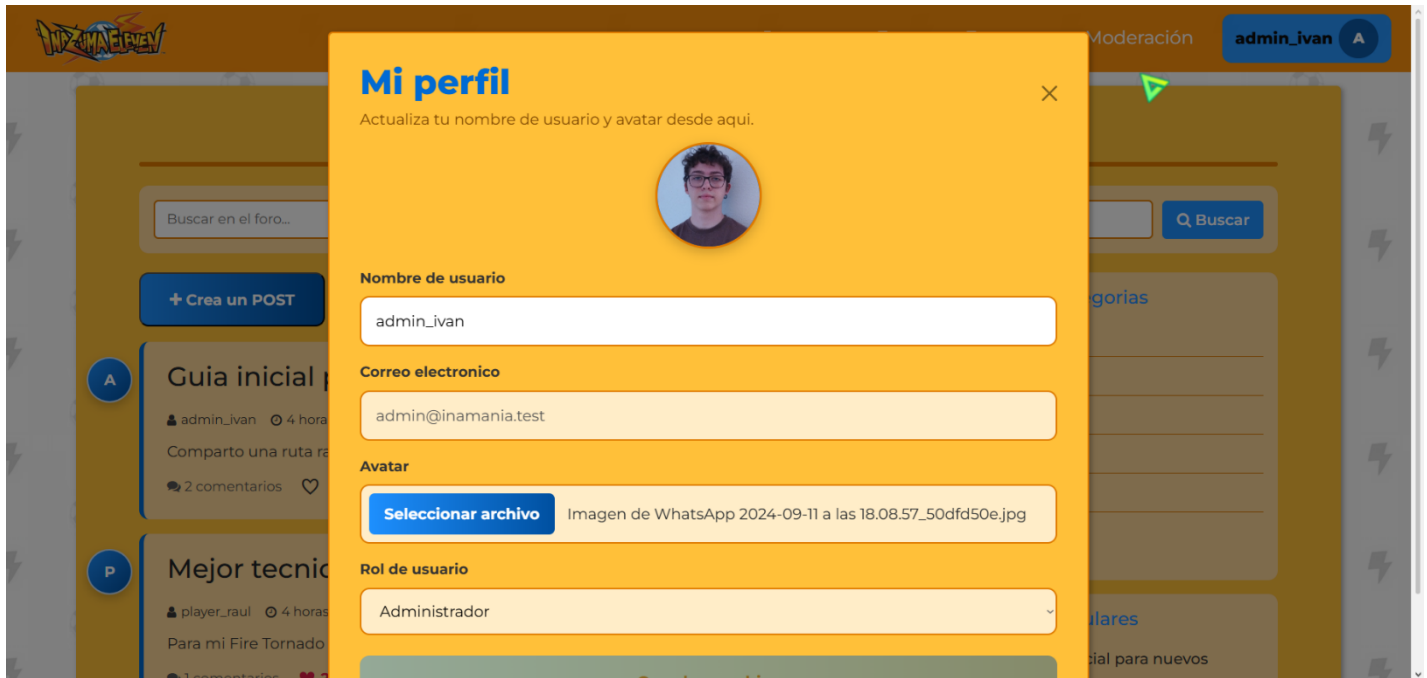
The screenshot shows a modal window titled "INAMANIA" with the subtitle "Accede a la comunidad o crea tu cuenta en unos segundos." It features two buttons at the top: "Iniciar sesión" (highlighted in blue) and "Registrarse". Below these, a section titled "Bienvenido de nuevo" says "Entra para participar en el foro, eventos y contenido exclusivo." It contains two input fields: "Correo electrónico" with the placeholder "tu@email.com" and "Contraseña" with a masked password "*****" and an eye icon to toggle visibility. At the bottom is a large blue button labeled "INICIAR SESIÓN". The background shows a blurred website header with "Log in" and navigation links "Juegos", "Foro", and "Eventos".

The screenshot shows a modal window titled "INAMANIA" with the subtitle "Accede a la comunidad o crea tu cuenta en unos segundos." It features two buttons at the top: "Iniciar sesión" and "Registrarse" (highlighted in blue). Below these, a section titled "Crea tu cuenta" says "Únete a la comunidad para comentar, seguir eventos y guardar favoritos." It contains four input fields: "Nombre de usuario" with the placeholder "Ej: MarkEvans10", "Correo electrónico" with the placeholder "tu@email.com", "Contraseña" with the placeholder "Mínimo 6 caracteres" and an eye icon, and "Confirmar contraseña" with the placeholder "Repite tu contraseña" and an eye icon. At the bottom is a large blue button labeled "CREAR CUENTA". The background shows a blurred website header with "Log in" and navigation links "Foro" and "Eventos".

Cuando se realiza una acción correctamente aparecerá temporalmente una notificación confirmando con color verde y cuando falla igual pero con rojo.

The screenshot shows the website header after a successful login. It features a green notification box on the left that says "Sesión iniciada correctamente." To the right of the notification are the links "Eventos" and "Moderación", and a user profile button labeled "admin_ivan" with a small "A" icon. Below the header is a large orange banner that says "Bienvenido a INAMANIA".

Cada usuario tiene un perfil en el que puede cambiar su nombre y su avatar.



En juegos podemos observar que salen dinámicamente los datos asociados a la base de datos para que un moderador pueda cambiarlo rápidamente.

[Juegos](#)[Foro](#)[Eventos](#)[Moderación](#)

admin_ivan

Saga de Juegos de Inazuma Eleven

Inazuma Eleven



Inazuma Eleven

La aventura del Raimon para llegar al Football Frontier.



Inazuma Eleven 2 Tormenta de Fuego

Segunda entrega de la saga original con nuevos rivales y técnicas.

Inazuma Eleven GO



Inazuma Eleven GO Luz

Tenma y sus compañeros luchan por recuperar el futbol.

Contacto

Email: info@inamania.com
Teléfono: +34 123 456 789

Legal

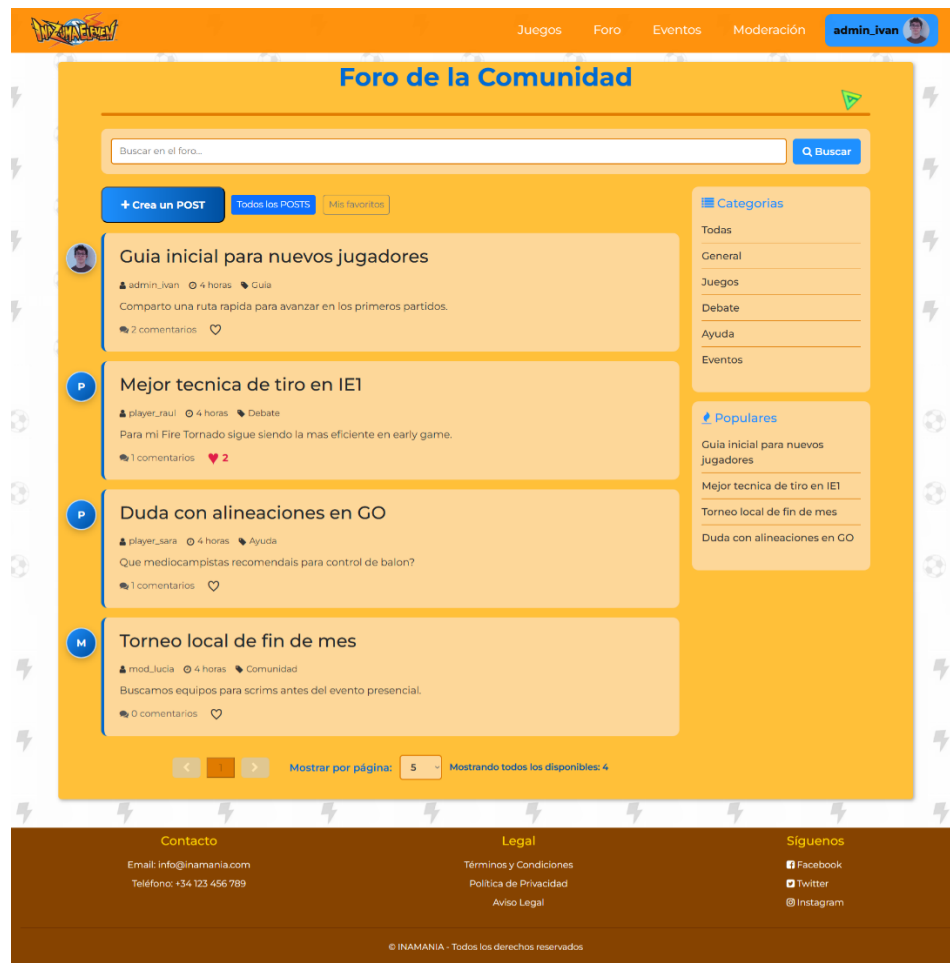
[Términos y Condiciones](#)
[Política de Privacidad](#)
[Aviso Legal](#)

Síguenos

 [Facebook](#)
 [Twitter](#)
 [Instagram](#)

© INAMANIA - Todos los derechos reservados

En el apartado del foro podemos observar cada post como una tarjeta con un buscador que filtra los posts, además de poder filtrar usando la categoría o ver los posts más populares que se ordenan de manera descendente por el numero de favoritos, para poder hacer uso de crear, comentar o dar a favoritos es necesario que el usuario este logueado.



En eventos podemos ver un calendario dinamico en el que aparecen los eventos futuros y hay 4 opciones, la primera es que sea de pago y se añada al carrito, la segunda es que sea gratuito, la tercera que el plazo de inscripción aun no haya empezado y que ya estes inscrito.


[Juegos](#)
[Foro](#)
[Moderación](#)
[admin_ivan](#)

Eventos y Torneos

<

Mayo 2026

>

Puedes navegar entre los proximos 12 meses.

Lun	Mar	Mie	Jue	Vie	Sab	Dom
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

Eventos - 15 Mayo 2026
 No hay eventos para este día.

Eventos recomendados

JUN 04

Bootcamp Tactico Premium
 Entrenamiento avanzado con analisis de partidos.
 Inscritos: 1

25,00 €

MAY 22

Open Day Comunidad
 Jornada abierta sin limite de plazas.
 Inscritos: 0

UNIRME

JUN 02

Clínica de Porteros
 Sesión especializada para posicion POR.
 Inscritos: 0

APERTURA DE INSCRIPCIONES: 20/05/2026

Eventos a los que estas inscrito

MAY 20

Copa Relampago
 Formato rapido con plazas limitadas.

DESINSCRIBIRME

MAY 25

Liga Escolar Abierta
 Evento gratuito para equipos novatos.

DESINSCRIBIRME

Información Importante

Cómo Participar
 Regístrate, explora el calendario y confirma tu inscripción en el evento deseado.

Sistema de Premios
 Los premios varían según el evento e incluyen merchandising, códigos y reconocimientos.

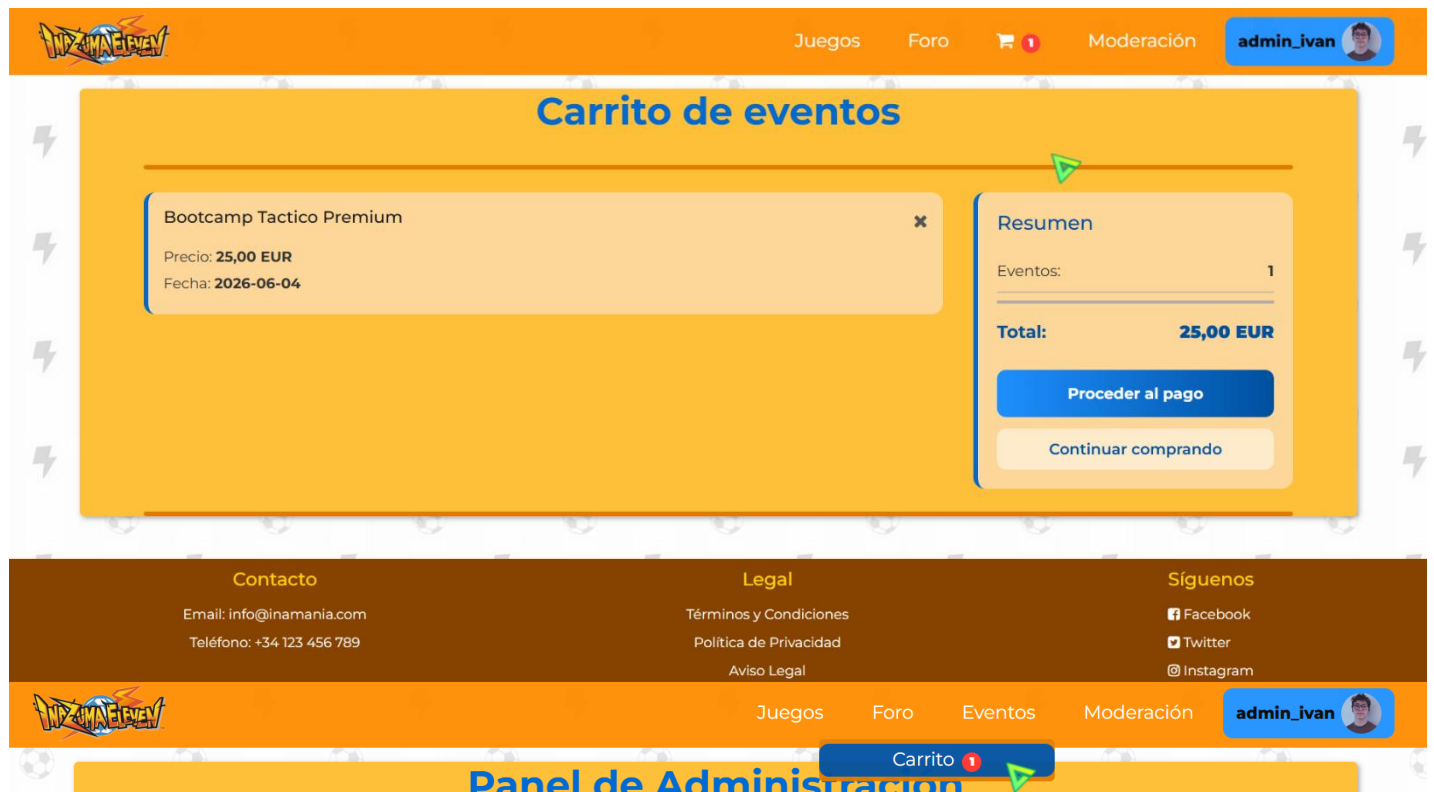
Contacto
 Email: info@inamania.com
 Teléfono: +34 123 456 789

Legal
 Términos y Condiciones
 Política de Privacidad
 Aviso Legal

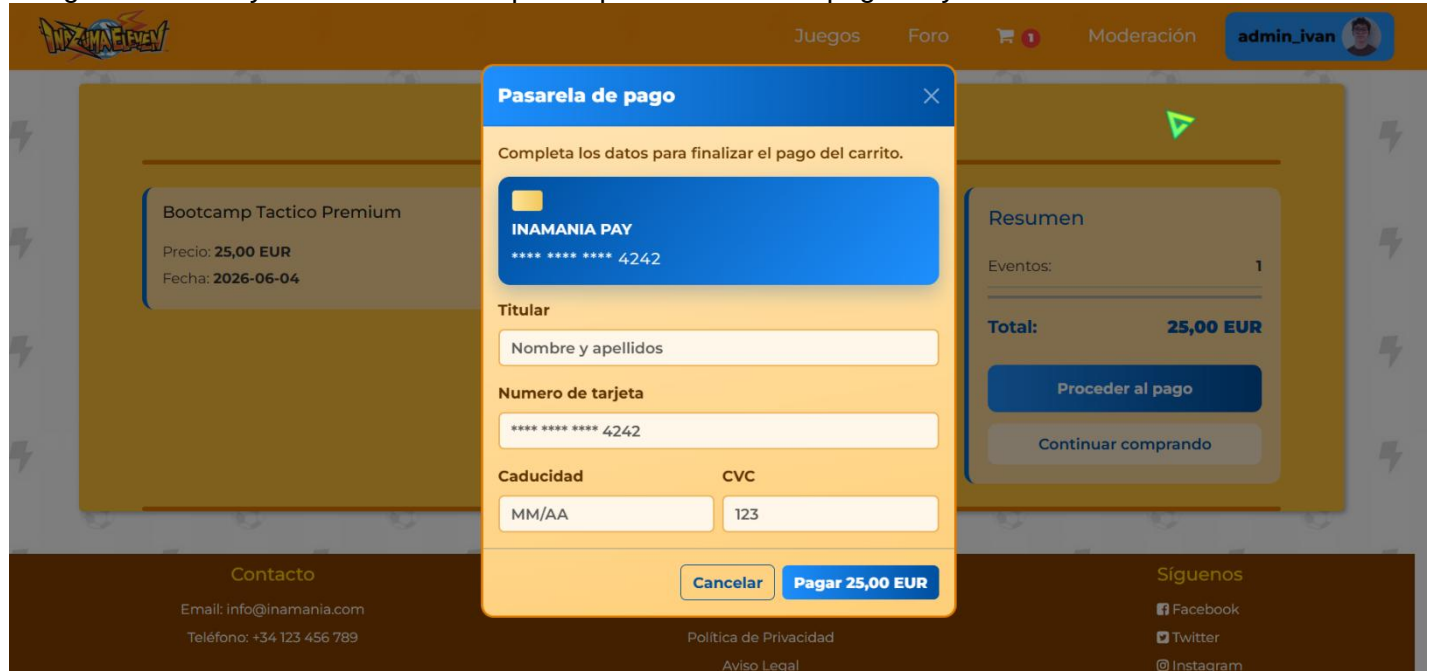
Síguenos
[Facebook](#)
[Twitter](#)
[Instagram](#)

© INAMANIA - Todos los derechos reservados

Cuando le das a un evento de pago este se añade al carrito así.



La pasarela de pago no cobra realmente, pero hace una comprobación de que la tarjeta insertada cumple el código luhn y si lo cumple procede el pago y te inscribirías al evento



En la guía de un juego concreto lo más relevante es el buscador de equipo
llegas a una coincidencia se limitarán los personajes a aquellos que tengan
vinculados la coincidencia e igual con el objeto coleccionable.

El efecto mencionado anteriormente funciona igual con personaje, cuando
tiene una coincidencia sale el equipo asociado y las supertecnica que tiene ese
personaje.

Con supertecnica se limitaran los personajes a los que tienen esa supertecnica
y los objetos es un simple buscador de los coleccionables

para funcionar como filtro.

se caracteriza principalmente por el sistema de filtros mediante la barra de
búsqueda haciéndolo más dinámico.

The screenshot displays the 'Inazuma Eleven' website interface, which is organized into several sections for user navigation and data viewing.

- Header:** Features the site logo, navigation links (Inicio, Noticias, Equipos, Personajes, Supertécnicas, Objetos Coleccionables), and a search bar.
- Capítulos de la Historia:** A section listing game chapters, such as 'Capítulo 1: Comienza la aventura' and 'Capítulo 2: El primer torneo de la Royal Academy'.
- Equipos:** A section dedicated to team profiles. It includes:
 - Raimon:** Described as 'el equipo original de la serie original', listing players like Raimon, Mark Evans, and others, along with their positions and stats.
 - Royal Academy:** Described as 'el equipo con los mejores jugadores', listing players like Jude Sharp and others, along with their positions and stats.
- Personajes:** A section for individual player profiles. It includes:
 - Avril Blaze:** A goalkeeper for Raimon, listing his stats (PS, DEF, ATA, etc.) and special abilities.
 - Jude Sharp:** A player for Royal Academy, listing his stats and special abilities.
 - Mark Evans:** A player for Raimon, listing his stats and special abilities.
- Supertécnicas:** A section for special techniques. It includes:
 - Fuero Tornado:** A technique used by Avril Blaze, listing its stats and effects.
 - Goal Hand:** A technique used by Jude Sharp, listing its stats and effects.
- Objetos Coleccionables:** A section for collectible items. It includes:
 - Escudo Raimon:** A collectible item for Raimon, listing its stats and effects.
 - Escudo Royal Academy:** A collectible item for Royal Academy, listing its stats and effects.
 - Foto del Raimon Inicial:** A collectible item representing the initial Raimon team photo, listing its stats and effects.

Existen dos paneles de administración, el de contenido y el de moderación, uno es para controlar la parte de home y las guías mientras que el otro es para controlar los usuarios, el foro y los eventos.

[Juegos](#)
[Foro](#)
[Eventos](#)
[Moderación](#)

admin_ivan
[Favoritos](#)
[Eventos](#)
[Perfil](#)
[Cerrar sesión](#)

Panel de Administración

Sagas

Nombre * Descripción

[Crear Saga](#)

ID	Nombre	Descripción	Acciones
1	Inazuma Eleven	Saga original de Inazuma Eleven.	Inazuma Eleven Editar Borrar
2	Inazuma Eleven GO	Arco GO con nueva generacion de jugadores.	Inazuma Eleven GO Editar Borrar

Plataformas

Nombre *

[Crear Plataforma](#)

ID	Nombre	Acciones
2	Nintendo 3DS	Nintendo 3DS Editar Borrar
1	Nintendo DS	Nintendo DS Editar Borrar

Juegos

Título * Saga Plataforma

Año Descripción Imagen Ningún archivo seleccionado

[Crear Juego](#)

ID	Título	Saga	Plataforma	Año	Acciones
1	Inazuma Eleven	Inazuma Eleven	Nintendo DS	2008	Borrar
3	Inazuma Eleven 2 Tormenta de Fuego	Inazuma Eleven	Nintendo DS	2009	Borrar
2	Inazuma Eleven GO Luz	Inazuma Eleven GO	Nintendo 3DS	2011	Borrar

Personajes

Juego * Nombre * Posición *

Elemento * Descripción Estilo de Juego

Imagen Ningún archivo seleccionado

Estadísticas (opcional)

Pe	DT	Tiro	Control	Defensa	Rapidez	Físico
Aguante						

[Crear Personaje](#)

ID	Nombre	Juego	Pos	Elem	Acciones
2	Avel Blaze	Inazuma Eleven	DL	Fuego	Borrar
3	Jude Sharp	Inazuma Eleven	MD	Aire	Borrar
1	Mark Evans	Inazuma Eleven	POR	Montaña	Borrar
4	Arlon Sherwind	Inazuma Eleven GO Luz	MD	Aire	Borrar
5	Victor Blade	Inazuma Eleven GO Luz	DL	Fuego	Borrar

Equipos

Juego * Nombre * Descripción

Entrenador Uniforme Estilo de Juego

Escudo (Imagen) Ningún archivo seleccionado

[Crear Equipo](#)

ID	Nombre	Juego	Entrenador	Acciones
1	Raimon	Inazuma Eleven	Nelly Raimon	Borrar
2	Royal Academy	Inazuma Eleven	Jude Sharp	Borrar
3	Raimon GO	Inazuma Eleven GO Luz	Riccardo Di Rigo	Borrar

Panel de Moderación

Usuarios: bloquear/desbloquear

ID	Nombre	Avatar	Acción	Acción
1	admin_ivan		Borrar	Borrar
2	admin_ivan		Borrar	Borrar
3	admin_ivan		Borrar	Borrar
4	admin_ivan		Borrar	Borrar
5	admin_ivan		Borrar	Borrar

Bloqueo por IP

IP

[Bloquear IP](#)

ID	Nombre	Acción
1	192.168.1.1	Borrar
2	192.168.1.1	Borrar

Foro: borrar posts y comentarios

ID	Título	Author	Acción	Post	Author	Acción
1	hola mundo	admin_ivan	Borrar	1	admin_ivan	Borrar
2	hola mundo	admin_ivan	Borrar	2	admin_ivan	Borrar
3	hola mundo	admin_ivan	Borrar	3	admin_ivan	Borrar
4	hola mundo	admin_ivan	Borrar	4	admin_ivan	Borrar
5	hola mundo	admin_ivan	Borrar	5	admin_ivan	Borrar

Eventos: crear y modificar

Título Descripción

Evento Fecha

[Crear Evento](#)

Home: carrusel

Título Descripción

[Crear Carrusel](#)

Home: noticias

Título Descripción

[Crear Noticia](#)

Contacto
 Email: info@inamania.com
 Teléfono: +34 123 456 789

Legal
 Términos y Condiciones
 Política de Privacidad
 Aviso Legal

Síguenos
[Facebook](#)
[Twitter](#)
[Instagram](#)

© INAMANIA - Todos los derechos reservados

3.2.2.3.2. Accesibilidad

La accesibilidad ha sido uno de los pilares principales durante el desarrollo de INUMANIA, con el objetivo de que cualquier usuario pueda navegar e interactuar con la plataforma independientemente de sus capacidades físicas o del dispositivo utilizado. Para ello se han seguido las recomendaciones de las pautas WCAG 2.2.

Medidas de accesibilidad implementadas

- **Diseño responsive y adaptable**
Toda la interfaz se adapta automáticamente a móviles, tablets y ordenadores mediante Bootstrap y media queries, garantizando una correcta visualización del contenido en cualquier resolución.
- **Contraste de colores adecuado**
La combinación de colores principal (#FF8C00, #1E90FF y fondos claros) se ha seleccionado buscando una buena legibilidad del texto y diferenciación visual entre elementos interactivos.
- **Navegación mediante teclado**
Los formularios, botones, enlaces y elementos interactivos pueden utilizarse mediante teclado usando la tecla TAB, facilitando el acceso a usuarios con dificultades motoras.
- **Etiquetas y validaciones en formularios**
Todos los formularios cuentan con labels descriptivos, placeholders y mensajes de error claros para ayudar al usuario a identificar los datos requeridos.
- **Uso de texto alternativo en imágenes**
Las imágenes importantes, como logos, personajes o carátulas, incorporan atributos alt descriptivos para lectores de pantalla.
- **Jerarquía semántica HTML**
Se han utilizado etiquetas semánticas como header, main, section, article, nav y footer, mejorando tanto la accesibilidad como el posicionamiento SEO.
- **Feedback visual inmediato**
Cuando el usuario realiza acciones importantes (registro, login, errores o confirmaciones), el sistema muestra notificaciones visuales claras mediante colores diferenciados:
 - Verde → acción correcta.
 - Rojo → error o validación incorrecta.
- **Tamaño de elementos interactivos**
Los botones y enlaces tienen dimensiones adecuadas para facilitar su pulsación tanto en pantallas táctiles como con ratón.
- **Comprobación wave**
Todas las paginas han sido probadas usando wave y dando un resultado mínimo de 5 puntos en accesibilidad.

3.2.2.3.3. Usabilidad

La usabilidad de INUMANIA se ha diseñado para ofrecer una experiencia intuitiva, rápida y sencilla, permitiendo que cualquier usuario encuentre información o interactúe con la comunidad sin dificultad.

Aspectos de usabilidad implementados

- **Interfaz intuitiva y sencilla**

La estructura de navegación es clara y está organizada en apartados principales:

- Inicio
- Juegos
- Foro
- Eventos
- Perfil

Esto permite que el usuario encuentre rápidamente la sección deseada.

- **Sistema de navegación coherente**

Todos los menús mantienen la misma estructura visual y comportamiento en toda la aplicación, evitando confusiones.

- **Buscadores y filtros dinámicos**

Las guías de videojuegos incluyen sistemas de búsqueda avanzada y filtros relacionados entre personajes, equipos, técnicas y objetos, mejorando enormemente la rapidez para localizar información específica.

- **Minimización de pasos innecesarios**

Acciones frecuentes como comentar, guardar favoritos o apuntarse a eventos requieren el menor número posible de clics.

- **Retroalimentación inmediata al usuario**

El sistema informa constantemente del resultado de las acciones realizadas:

- Confirmación de registros.
- Mensajes de error.
- Inscripciones realizadas.
- Publicaciones creadas correctamente.

- **Consistencia visual**

Toda la aplicación mantiene la misma guía de estilos, colores, tipografías y componentes, facilitando el aprendizaje de uso.

- **Optimización del rendimiento**

Se ha priorizado la carga rápida de páginas y consultas mediante contenido dinámico y consultas optimizadas a la base de datos.

- **Adaptación al usuario objetivo**

El diseño visual está inspirado en la estética de Inazuma Eleven para conectar directamente con la comunidad fan, pero manteniendo una interfaz limpia y no sobrecargada.

- **Paneles diferenciados por roles**

Administradores y moderadores disponen de paneles específicos adaptados a sus funciones, simplificando la gestión de contenido y usuarios.

- **Prevención de errores**

Los formularios validan datos antes de enviarlos al servidor para evitar errores comunes como:

- Correos inválidos.
- Campos vacíos.
- Contraseñas demasiado cortas.
- Datos incorrectos en pagos o eventos.

3.2.2.3.4. Desarrollo web entorno cliente

a. Formularios y su validación

Archivo	Formulario	Validación HTML	Validación JS
admin.php	Actualizar plataforma	Obligatorios: plataforma_nombre plataforma_nombre (text, required)	Sin validación JS específica.
admin.php	Actualizar saga	Obligatorios: saga_nombre saga_nombre (text, required)	Sin validación JS específica.
admin.php	Crear equipo	Obligatorios: equipo_juego_id, equipo_nombre multipart/form-data equipo_juego_id (select, required); equipo_nombre (text, required); equipo_descripcion (textarea); equipo_entrenador (text); equipo_uniforme (text); equipo_estilo (text); equipo_escudo (file, image/*)	Sin validación JS específica.
admin.php	Crear juego	Obligatorios: juego_titulo multipart/form-data juego_titulo (text, required); juego_saga_id (select); juego_plataforma_id (select); juego_ano (number, min=1900); juego_descripcion (textarea); juego_imagen (file, image/*)	Sin validación JS específica.
admin.php	Crear personaje	Obligatorios: personaje_juego_id, personaje_nombre, personaje_posicion, personaje_elemento multipart/form-data personaje_juego_id (select, req); personaje_nombre (text, req); personaje_posicion (select, req); personaje_elemento (select, req); personaje_descripcion (textarea); personaje_estilo (text); personaje_imagen (file, image/*); stats: pe, pt, tiro, control, defensa, rapidez, fisico, aguante (number, min=0)	Sin validación JS específica.
admin.php	Crear plataforma	Obligatorios: plataforma_nombre plataforma_nombre (text, required)	Sin validación JS específica.
admin.php	Crear saga	Obligatorios: saga_nombre saga_nombre (text, required); saga_descripcion (textarea)	Sin validación JS específica.
admin.php	Eliminar equipo	onsubmit inline: return confirm()	Solo confirmación inline con confirm().
admin.php	Eliminar juego	onsubmit inline: return confirm()	Solo confirmación inline con confirm().
admin.php	Eliminar personaje	onsubmit inline: return confirm()	Solo confirmación inline con confirm().
admin.php	Eliminar plataforma	onsubmit inline: return confirm()	Solo confirmación inline con confirm().
admin.php	Eliminar saga	onsubmit inline: return confirm()	Solo confirmación inline con confirm().
carrito.php	Checkout de pago	Obligatorios: checkoutCardName, checkoutCardNumber, checkoutCardExpiry, checkoutCardCvc novalidate checkoutCardName (text, req); checkoutCardNumber (text, req, maxlength=19); checkoutCardExpiry (text, req, maxlength=5); checkoutCardCvc (text, req, maxlength=4)	carrito.js: formatea número y caducidad; limita CVC a dígitos; valida titular ≥3 chars, tarjeta con Luhn, caducidad MM/AA no vencida y CVC 3-4 dígitos; marca is-valid/is-invalid; bloquea envío si falla.
carrito.php	Eliminar item del carrito	Sin atributos de validación HTML destacables.	carrito.js: intercepta submit por AJAX y exige que

Proyecto-intermodular de Ivan Martinez Navarro

Archivo	Formulario	Validación HTML	Validación JS
			exista evento_id antes de enviar.
foro.php	Búsqueda del foro	q (text)	foro.js: evita submit tradicional, normaliza texto quitando mayúsculas y acentos, filtra posts en tiempo real.
foro.php	Crear comentario	Obligatorios: comment_content comment_content (textarea, required, maxlength=1000)	Sin validación JS específica.
foro.php	Crear post del foro	Obligatorios: title, category, content title (text, req, maxlength=100); category (select, req); content (textarea, req)	Sin validación JS específica.
foro.php	Favorito de post	Sin atributos de validación HTML destacables.	Sin validación JS específica.
juego.php	Búsqueda de equipos wiki	search-equipo (search)	wiki-personajes.js: evita submit, normaliza texto, filtra tarjetas en tiempo real, pagina resultados y activa filtros cruzados si la búsqueda deja un único resultado.
juego.php	Búsqueda de objetos wiki	search-objeto (search)	wiki-personajes.js: evita submit, normaliza texto, filtra tarjetas en tiempo real, pagina resultados y activa filtros cruzados si la búsqueda deja un único resultado.
juego.php	Búsqueda de personajes wiki	search-personaje (search)	wiki-personajes.js: evita submit, normaliza texto, filtra tarjetas en tiempo real, pagina resultados y activa filtros cruzados si la búsqueda deja un único resultado.
juego.php	Búsqueda de supertécnicas wiki	search-supertecnica (search)	wiki-personajes.js: evita submit, normaliza texto, filtra tarjetas en tiempo real, pagina resultados y activa filtros cruzados si la búsqueda deja un único resultado.
login.php	Formulario genérico	Obligatorios: email, password email (email, req); password (password, req) Obligatorios: username, email, password, password_confirmation username (text, req); email (email, req); password (password, req, minlength=6); password_confirmation (password, req, minlength=6)	Sin validación JS específica.
login-modal.php	Inicio de sesión	Obligatorios: email, password email (email, req); password (password, req)	auth-modal.js: no valida el envío; solo alterna pestañas del modal y muestra/oculta contraseñas.
login-modal.php	Registro de usuario	Obligatorios: username, email, password,	auth-modal.js: no valida

Proyecto-intermodular de Ivan Martinez Navarro

Archivo	Formulario	Validación HTML	Validación JS
		password_confirmation username (text, req, maxlength=16); email (email, req); password (password, req, minlength=6); password_confirmation (password, req, minlength=6)	campos al enviar; solo alterna pestañas del modal y muestra/oculta contraseñas.
moderacion.php	Actualizar evento	Obligatorios: title, start_date title (text, req); description (textarea); start_date (date, req); end_date (date); registration_date (date); location (text); slots (number, min=1, step=1); price (number, min=0, step=0.01)	Sin validación JS específica.
moderacion.php	Actualizar noticia	Obligatorios: title, text multipart/form-data title (text, req); text (textarea, req); image (file, image/*); order (number, min=1, step=1); active (checkbox)	Sin validación JS específica.
moderacion.php	Actualizar slide de carrusel	Obligatorios: title multipart/form-data title (text, req); description (textarea); image (file, image/*); order (number, min=1, step=1); active (checkbox)	Sin validación JS específica.
moderacion.php	Bloquear IP	Obligatorios: ip ip (text, req); reason (text, maxlength=255); blocked_until (datetime-local)	Sin validación JS específica.
moderacion.php	Bloquear usuario	reason (text, maxlength=255); blocked_until (datetime-local)	Sin validación JS específica.
moderacion.php	Cambiar rol de usuario	Obligatorios: new_role new_role (select, req)	Sin validación JS específica.
moderacion.php	Crear evento	Obligatorios: title, start_date title (text, req); description (textarea); start_date (date, req); end_date (date); registration_date (date); location (text); slots (number, min=1, step=1); price (number, min=0, step=0.01)	Sin validación JS específica.
moderacion.php	Crear noticia	Obligatorios: title, text, image multipart/form-data title (text, req); text (textarea, req); image (file, req, image/*); order (number, min=1, step=1); active (checkbox)	Sin validación JS específica.
moderacion.php	Crear slide de carrusel	Obligatorios: title, image multipart/form-data title (text, req); description (textarea); image (file, req, image/*); order (number, min=1, step=1); active (checkbox)	Sin validación JS específica.
moderacion.php	Desbloquear IP	Sin atributos de validación HTML destacables.	Sin validación JS específica.
moderacion.php	Desbloquear usuario	Sin atributos de validación HTML destacables.	Sin validación JS específica.
moderacion.php	Eliminar comentario	onsubmit inline: return confirm()	Solo confirmación inline con confirm().
moderacion.php	Eliminar post	onsubmit inline: return confirm()	Solo confirmación inline con confirm().
navbar.php	Cierre de sesión	Sin atributos de validación HTML destacables.	Sin validación JS específica.
perfil-modal.php	Actualizar perfil	Obligatorios: username multipart/form-data username (text, req, maxlength=16); profile-modal-email (email, readonly, disabled); avatar (file, image/jpeg, image/png, image/webp, image/gif); rol (select)	profile-modal.js: al cambiar avatar, previsualiza el archivo si su MIME empieza por image/.

b. Manejo y gestión de eventos: Teclado, ratón y estados de la ventana

Archivo	Elemento	Implementación	Descripción
auth-modal.js	Toggle de contraseña	click sobre .password-toggle	Alterna entre mostrar y ocultar la contraseña del formulario.
auth-modal.js	Apertura del modal de autenticación	show.bs.modal en #authModal	Selecciona la pestaña correcta al abrir el modal según el disparador.
auth-modal.js	Disparadores del modal	click sobre [data-auth-trigger="modal"]	Abre el modal y fija el modo login o registro.
auth-modal.js	Cambio manual de pestañas	click sobre [data-auth-tab]	Permite cambiar entre login y registro dentro del modal.
carrito.js	Agregar evento al carrito	click sobre .add-to-cart-btn	Captura la pulsación del botón y lanza el alta del evento.
carrito.js	Eliminar evento del carrito	submit sobre .remove-form	Intercepta el envío del formulario para borrarlo por JavaScript.
carrito.js	Inscripción gratuita	click sobre .join-free-event-btn	Gestiona la inscripción del usuario en eventos gratuitos.
carrito.js	Desinscripción de evento	click sobre .leave-event-btn	Gestiona la baja del usuario en un evento.
carrito.js	Formato de número de tarjeta	input sobre #checkoutCardNumber	Reformatea el número introducido y actualiza la vista previa.
carrito.js	Formato de caducidad	input sobre #checkoutCardExpiry	Inserta automáticamente la barra MM/AA.
carrito.js	Restricción del CVC	input sobre #checkoutCardCvc	Limita la entrada a dígitos y longitud máxima.
carrito.js	Envío del checkout	submit sobre #checkoutGatewayForm	Valida el formulario de pago y lanza el proceso de cobro.
custom-cursor.js	Seguimiento del ratón	pointermove y mousemove sobre document	Actualiza la posición del cursor personalizado en tiempo real.
custom-cursor.js	Salida del puntero	mouseleave sobre document	Oculto el cursor personalizado cuando el puntero abandona el documento.
custom-cursor.js	Estados de la ventana	blur y focus sobre window	Oculto o muestra el cursor según el foco de la ventana.
custom-cursor.js	Salida real de la ventana	mouseout sobre window con relatedTarget === null	Detecta cuando el ratón sale realmente de la ventana del navegador.
eventos-calendar.js	Selección de día	click sobre botones de día generados dinámicamente	Cambia el día seleccionado y vuelve a renderizar el calendario.
eventos-calendar.js	Navegación al mes anterior	click sobre [data-calendar-prev]	Mueve la vista del calendario un mes hacia atrás.
eventos-calendar.js	Navegación al mes siguiente	click sobre [data-calendar-next]	Mueve la vista del calendario un mes hacia adelante.
foro.js	Desplegar comentarios	click sobre .comments-toggle-btn	Muestra u oculta el panel de comentarios de cada publicación.
foro.js	Desplegar formulario de post	click sobre #toggle-create-post	Muestra u oculta el panel de creación de publicaciones.
foro.js	Búsqueda en tiempo real	input sobre [data-forum-search-input]	Refiltra los posts del foro mientras el usuario escribe.
foro.js	Búsqueda del foro por submit	submit sobre [data-forum-search-form]	Evita el envío tradicional y aplica el filtro en cliente.
profile-	Vista previa de avatar	change sobre #profile-modal-	Lee la imagen seleccionada y actualiza la

Proyecto-intermodular de Ivan Martinez Navarro

Archivo	Elemento	Implementación	Descripción
modal.js		avatar	previsualización.
profile-modal.js	Apertura del modal de perfil	click sobre [data-profile-trigger="modal"]	Abre el modal de edición de perfil.
wiki-personajes.js	Búsqueda en navegadores wiki	input sobre los buscadores de cada sección	Actualiza resultados y relaciones cruzadas mientras se escribe.
wiki-personajes.js	Búsqueda por formulario	submit sobre los formularios de búsqueda wiki	Evita la recarga y aplica el filtro en cliente.
wiki-personajes.js	Paginación dinámica	click sobre botones de paginación generados	Avanza o retrocede entre páginas de resultados.
General	Eventos de teclado	No detectado en JS propio	No hay listeners específicos de teclado como keydown o keyup en el código JavaScript del proyecto.

c. Gestión y almacenamiento de datos e información en el cliente

Archivo	Elemento	Implementación	Descripción
carrito.js	Empaquetado de datos para peticiones	FormData	Construye objetos FormData para enviar identificadores y metadatos de eventos al servidor.
carrito.js	Recogida de datos del DOM	getAttribute y campos hidden	Lee data-evento-id, data-evento-titulo, data-evento-precio, data-evento-fecha e inputs ocultos para operar con los eventos.
carrito.js	Validación y estado temporal del pago	Variables locales y funciones de validación	Calcula y conserva en memoria el estado válido o inválido de nombre, tarjeta, caducidad y CVC.
custom-cursor.js	Estado del cursor personalizado	Variables isEnabled, hasPosition, preloaded	Mantiene en memoria el estado visual y de precarga del cursor personalizado.
custom-cursor.js	Configuración desde CSS	getComputedStyle y parseo de --asset-cursor-url	Obtiene información del cliente a partir de una variable CSS para decidir si cargar la imagen del cursor.
eventos-calendar.js	Carga de datos embebidos	JSON.parse sobre #calendar-events-data	Convierte a objetos JavaScript los datos JSON enviados por el servidor dentro del HTML.
eventos-calendar.js	Agrupación de eventos por fecha	Objeto eventsByDate	Reorganiza los eventos en memoria para acceder rápidamente por día.
eventos-calendar.js	Estado del calendario	Variables currentViewMonth y selectedDate	Conserva el mes mostrado y el día activo mientras el usuario navega.
foro.js	Índice de búsqueda de publicaciones	Atributo data-search normalizado	Compara en cliente el texto buscado con una cadena de datos preparada en cada tarjeta.
profile-modal.js	Lectura local de archivos	FileReader.readAsDataURL	Convierte temporalmente la imagen seleccionada en una URL base64 para previsualizarla sin subirla aún.
wiki-personajes.js	Normalización de cadenas	Función normalize	Transforma texto para comparar sin depender de mayúsculas o tildes.
wiki-personajes.js	Conjuntos de relaciones	Set generado por parseIdSet	Almacena IDs relacionados para aplicar filtros cruzados entre equipos, personajes, técnicas y objetos.
wiki-personajes.js	Estado de cada navegador	Array states y objeto statesById	Guarda en memoria paginación, filtros activos, tarjetas coincidentes y referencias al DOM.
wiki-personajes.js	Filtro relacional externo	externalFilterIds	Conserva el subconjunto de IDs permitidos cuando una sección condiciona a las demás.
General	Almacenamiento persistente	No detectado en JS propio	No hay usos de localStorage, sessionStorage, cookies, IndexedDB ni caché persistente en el JavaScript del proyecto.

d. Modificación del DOM

Archivo	Elemento	Implementación	Descripción
auth-modal.js	Cambio de pestañas del modal	classList.toggle y setAttribute	Activa o desactiva pestañas y paneles del modal de autenticación modificando clases y atributos ARIA.
carrito.js	Creación de notificaciones	createElement, appendChild, textContent y remove	Inserta alertas dinámicas en el body y las elimina tras mostrarlas.
carrito.js	Marcado visual de validación	classList.toggle sobre is-valid e is-invalid	Modifica las clases de los campos del formulario de pago según su validez.
carrito.js	Actualización de vista previa	textContent sobre .gateway-card-number	Cambia el texto visible del número de tarjeta en la pasarela visual.
custom-cursor.js	Inserción del cursor personalizado	createElement, setAttribute, appendChild	Genera un nodo div para el cursor personalizado y lo inserta en el body.
custom-cursor.js	Visibilidad del cursor	classList.add y classList.remove	Activa y desactiva clases CSS que controlan la presencia del cursor.
eventos-calendar.js	Reinicio de rejilla y listado	innerHTML = "	Limpia contenedores antes de volver a pintar días y eventos.
eventos-calendar.js	Construcción de la rejilla	createElement, appendChild, className, setAttribute	Genera dinámicamente celdas vacías, botones de día e insignias con número de eventos.
eventos-calendar.js	Actualización de cabecera y detalle	textContent e innerHTML	Reescribe el título del mes y el contenido del panel de eventos del día seleccionado.
foro.js	Despliegue de paneles	removeAttribute, setAttribute, hidden y aria-expanded	Muestra u oculta comentarios y el formulario de crear post.
foro.js	Filtrado de publicaciones	Propiedad hidden sobre tarjetas y estado vacío	Oculta o muestra cada tarjeta del foro según el texto buscado.
flash-notification.js	Eliminación de mensajes	remove()	Suprime del DOM la notificación flash tras el tiempo de espera.
profile-modal.js	Renovación de la previsualización	remove, createElement, appendChild	Elimina la imagen previa o inicial y añade una nueva imagen seleccionada por el usuario.
wiki-personajes.js	Creación de contenedores de paginación	createElement, appendChild, setAttribute	Añade al DOM la estructura necesaria para paginar cada navegador wiki.
wiki-personajes.js	Regeneración de la paginación	innerHTML = " y botones dinámicos	Reconstruye los controles de página cada vez que cambian los resultados.
wiki-personajes.js	Filtrado visual de tarjetas	Propiedad hidden	Oculta o muestra tarjetas según página, filtro textual y filtro relacional.
wiki-personajes.js	Ocultación de barras de herramientas	Propiedad hidden sobre toolbar y emptyState	Adapta la interfaz cuando existen filtros cruzados o no hay resultados.

e. Animaciones, efectos, cambios dinámicos de estilos etc... para dinamizar la parte visible al cliente

Archivo	Elemento	Implementación	Descripción
auth-modal.js	Desvanecimiento de alertas temporales	style.transition, style.opacity y remove()	Aplica un fade out a los avisos del modal antes de eliminarlos.
auth-modal.js	Activación visual de pestañas	classList.toggle sobre active y show	Cambia dinámicamente el aspecto y el panel visible del modal de autenticación.
auth-modal.js	Apertura del modal	bootstrap.Modal.show()	Muestra el modal de autenticación como efecto visual controlado por Bootstrap.
carrito.js	Notificaciones flotantes	style.position, top, left, transform, zIndex, boxShadow y animation	Crea avisos emergentes centrados con animación de entrada y estilo dinámico.
carrito.js	Desaparición de notificaciones	style.transition y style.opacity	Realiza un desvanecimiento antes de retirar la notificación del DOM.
carrito.js	Feedback de validación	classList.toggle sobre is-valid e is-invalid	Marca visualmente los campos correctos o incorrectos durante el checkout.
carrito.js	Estado del botón de pago	disabled y textContent	Bloquea temporalmente el botón y cambia su texto a 'Procesando...' mientras dura la operación.
custom-cursor.js	Movimiento del cursor personalizado	style.left y style.top	Reposiciona visualmente el cursor personalizado según el movimiento del ratón.
custom-cursor.js	Mostrar u ocultar cursor	classList.add y classList.remove sobre is-visible	Activa o desactiva la visibilidad del cursor personalizado.
custom-cursor.js	Activación global del cursor	classList.add sobre custom-cursor-enabled	Aplica al documento el modo visual de cursor personalizado.
flash-notification.js	Desaparición diferida	setTimeout y remove()	Retira automáticamente la notificación flash tras unos segundos.
foro.js	Expandir y contraer paneles	hidden y aria-expanded	Cambia dinámicamente la visibilidad de comentarios y del formulario de publicación.
profile-modal.js	Apertura del modal de perfil	bootstrap.Modal.show()	Muestra el modal de edición de perfil como capa dinámica.
profile-modal.js	Actualización de avatar	Reemplazo de imagen previa en el DOM	Renueva la vista visible del avatar al seleccionar una imagen nueva.
wiki-personajes.js	Filtrado y paginación visual	hidden sobre tarjetas y toolbars	Actualiza de forma dinámica qué bloques se ven y cuáles quedan ocultos.
eventos-calendar.js	Resaltado de estados del calendario	classList.add sobre is-selected, has-event e is-today	Aplica estilos condicionales a los días según selección y contenido.

f. Comunicación AJAX

Archivo	Operación	Endpoint	Método	Datos enviados	Respuesta	Descripción
carrito.js	Agregar evento al carrito	?page=carrito-add	POST	FormData con evento_id, titulo, precio, fecha_inicio	JSON	Añade un evento de pago al carrito sin recargar inicialmente la página.
carrito.js	Eliminar evento del carrito	?page=carrito-remove	POST	FormData con evento_id	JSON	Elimina un evento del carrito por petición AJAX.
carrito.js	Inscripción gratuita	?page=evento-join-free	POST	FormData con evento_id	JSON	Inscribe al usuario en un evento gratuito mediante fetch.
carrito.js	Desinscripción de evento	?page=evento-leave	POST	FormData con evento_id	JSON	Da de baja al usuario de un evento sin usar envío tradicional.
carrito.js	Checkout de pago	?page=carrito-checkout	POST	Sin cuerpo adicional en la llamada final	JSON	Completa el pago del carrito y espera una respuesta JSON del servidor.
carrito.js	Identificación AJAX	Cabecera X-Requested-With	Incluida en POST	Cabecera XMLHttpRequest	No aplica	Marca las solicitudes para que el servidor pueda tratarlas como peticiones AJAX.

g. Comunicación asíncrona con el servidor

Archivo	Patrón asíncrono	Implementación	Descripción
carrito.js	Lanzamiento no bloqueante de solicitudes	fetch()	Envía peticiones al servidor sin bloquear la interfaz ni recargar inmediatamente la página.
carrito.js	Encadenado de respuesta	.then(...).then(...)	Procesa primero la respuesta HTTP y después el JSON devuelto por el servidor.
carrito.js	Tratamiento de errores	.catch(function (error) { ... })	Captura fallos de red o respuestas de error convertidas en excepción.
carrito.js	Limpieza de estado al finalizar	.finally(function () { ... })	Restaura el botón de pago, aunque la operación termine bien o mal.
carrito.js	Parseo asíncrono del cuerpo	response.json()	Deserializa de forma asíncrona el contenido JSON de la respuesta.
carrito.js	Coordinación con la UI	setTimeout después de éxito	Postpone la recarga de la página para mostrar antes la notificación al usuario.
carrito.js	Cierre del modal tras éxito	hide() después de respuesta correcta	Espera al resultado del servidor para cerrar la pasarela de pago.
profile-modal.js	Lectura asíncrona de archivos	FileReader.onload	Actualiza la vista previa del avatar cuando termina la lectura local del fichero.
custom-cursor.js	Precarga asíncrona de recursos	Image.onload, Image.onerror y setTimeout	Activa el cursor personalizado cuando finaliza o expira la carga de la imagen.
auth-modal.js	Temporización de efectos	setTimeout anidados	Programa la desaparición diferida de alertas temporales en el modal.
flash-notification.js	Temporización de retirada	setTimeout	Elimina de forma asíncrona la notificación flash después de un intervalo.

3.2.3. Fase de despliegue

3.2.3.1. Despliegue utilizando un hosting

El despliegue lo he realizado en la web de hostalia, esta me permite tener el hosting dns y dominios unificados, el primer paso es acceder al plan de hosting y una vez hecho registre mi dominio en la web para poder realizar después un posicionamiento de esta, una vez con nuestro dominio iremos a las configuraciones del dns.

☐	ftp.inamania.es	A	82.194.68.119	⋮
☐	imap.inamania.es	A	217.116.0.237	⋮
☐	inamania.es	TXT	v=spf1 redirect=spf.dominioabsoluto.net	⋮
☐	inamania.es	MX 10	mx.inamania.es.	⋮
☐	inamania.es	A	82.194.68.119	⋮
☐	mx.inamania.es	A	217.116.0.227	⋮
☐	pop3.inamania.es	A	217.116.0.237	⋮
☐	smtp.inamania.es	A	217.116.0.228	⋮
☐	webmail.inamania.es	A	217.116.0.155	⋮
☐	www.inamania.es	A	82.194.68.119	⋮

A continuación, lo que haremos será ir al hosting y añadir los archivos del proyecto, como uso php y Mysql no necesito instalar nada, si usara wordpress debería instalar el plugin.

Administrador de archivos para inamania.es

Directorio principal

- > .cagefs
- > .cl.selector
- > .clwpos
- > .trash
- > error_docs
- > httpdocs
- > inamania.es
- > logs
- > machapon.es
- > private

Directorio principal > inamania.es >

☐	Nombre ↑	Modificación	Tamaño	Permisos	Usuario	Grupo
☐	..	13/Mayo/2026 09:58		rwX --X ---	user-11367764	psaserv
☐	app	14/Mayo/2026 23:55		rwX r-X r-X	user-11367764	psacln
☐	config	14/Mayo/2026 23:55		rwX r-X r-X	user-11367764	psacln
☐	estilos	14/Mayo/2026 23:55		rwX r-X r-X	user-11367764	psacln
☐	uploads	14/Mayo/2026 23:55		rwX r-X r-X	user-11367764	psacln
☐	.htaccess	14/Mayo/2026 23:55	178 B	rw- r-- r--	user-11367764	psacln
☐	.user.ini	15/Mayo/2026 07:00	1.0 KB	rw- r-- r--	user-11367764	psacln
☐	index.php	14/Mayo/2026 23:55	67 B	rw- r-- r--	user-11367764	psacln

Como no puedo acceder directamente a la configuración de apache hice un .htacces en todas las carpetas con información sensible para que sean completamente inaccesibles(esto solo funciona si está en AllowOverride All).

Por último, lo que haremos es vincular una base de datos a nuestra web, en este caso remota y con credenciales.

11367764_inamania

Relacionadas con inamania.es

phpMyAdmin

Información de la conexión

Exportar volcado

Importar volcado

Copiar base de datos

Verificar y reparar

Host PMYSQL202.dns-servicio.com:3306 (MySQL) Usuario Ivanobich Tablas 20 Tamaño 624 KB

3.3. Seguimiento y control de incidencias

El seguimiento de la web se hará mediante los moderadores y el control de incidencias se llevaría acabo tanto por whatsapp, como llamadas al numero y por correos al gmail de soporte de la web.

Si observamos una crecida en la afluencia de usuarios en la página web se valoraría la opción de hacer un sistema de tickets.

3.4. Indicadores de calidad de procesos

Para garantizar que I funcione correctamente y mantenga un nivel alto de calidad tanto técnica como de experiencia de usuario, se han definido una serie de indicadores que permiten evaluar el rendimiento del proyecto durante el desarrollo y tras su despliegue.

Indicadores técnicos:

- **Tiempo de carga de páginas**
Se controla que las páginas principales carguen en menos de 3 segundos para evitar abandono de usuarios y mejorar la experiencia de navegación.
- **Disponibilidad del sistema**
El objetivo es mantener la aplicación disponible el mayor tiempo posible, minimizando caídas del servidor y errores críticos.
- **Número de errores detectados**
Se registran y corrigen errores encontrados durante las pruebas y el uso real de la plataforma para mejorar continuamente la estabilidad.

Indicadores de usabilidad:

- **Facilidad de navegación**
Se evalúa si los usuarios pueden acceder rápidamente a las diferentes secciones sin confusión.
- **Número de acciones completadas correctamente**
Se comprueba que procesos como registro, inicio de sesión, creación de posts o inscripción a eventos se completen sin incidencias.
- **Adaptabilidad responsive**
La aplicación se prueba en móviles, tablets y ordenadores para verificar que todos los elementos se muestran correctamente.
- **Accesibilidad**
Se revisa el cumplimiento de buenas prácticas WCAG, como navegación por teclado, contraste de colores y etiquetas descriptivas.

Indicadores de seguridad:

- **Protección de contraseñas**
Las contraseñas se almacenan cifradas mediante hash para evitar accesos no autorizados.
- **Control de usuarios bloqueados**
El sistema registra bloqueos de usuarios e IPs para prevenir abusos o comportamientos maliciosos.
- **Validación de formularios**
Todos los formularios validan los datos tanto en cliente como en servidor para reducir vulnerabilidades y errores.

Indicadores de mantenimiento y escalabilidad:

- **Facilidad de actualización**
Los datos de juegos, noticias y eventos pueden modificarse desde paneles de administración sin necesidad de alterar el código fuente.
- **Capacidad de ampliación**
La arquitectura permite añadir nuevos juegos, funcionalidades o idiomas en futuras versiones de la plataforma.

4. Recursos materiales

Para el desarrollo y mantenimiento de Inamania es necesario disponer de una serie de recursos materiales y tecnológicos que permitan llevar a cabo las tareas de análisis, programación, pruebas, despliegue y mantenimiento de la plataforma web.

Al tratarse de un proyecto de desarrollo web, los recursos empleados se centran principalmente en equipamiento informático, conexión a internet, herramientas de desarrollo y servicios de alojamiento web. La selección de estos recursos busca garantizar un entorno de trabajo estable, eficiente y económicamente viable para la puesta en marcha del proyecto.

Los medios utilizados han sido seleccionados teniendo en cuenta criterios de coste, rendimiento, compatibilidad con las tecnologías empleadas y posibilidades de ampliación futura.

4.1. Inventario, valorado de medios

La siguiente tabla recoge los principales recursos materiales y software utilizados durante el desarrollo de Inamania, junto con una estimación de su coste económico.

Recurso	Descripción	Coste estimado
Ordenador portátil	Equipo principal utilizado para el desarrollo y mantenimiento de la plataforma	1.180,00 €
Monitor adicional	Mejora de productividad durante las tareas de programación y diseño	180,00 €
Conexión a internet	Acceso a repositorios, documentación y servicios online	30,00 €/mes
Hosting web	Alojamiento de la aplicación y base de datos	93,00 €/año
Dominio web	Registro del dominio inamania.es	5,25 €/año
Visual Studio Code	Entorno de desarrollo integrado	Gratuito
XAMPP	Servidor local para pruebas y desarrollo	Gratuito
Bootstrap	Framework CSS para diseño responsive	Gratuito
Git y GitHub	Control de versiones y gestión del código fuente	Gratuito
Navegadores web	Chrome, Firefox y Edge para pruebas de compatibilidad	Gratuito
Figma y GIMP	Diseño de interfaces y recursos gráficos	Gratuito
Espacio de trabajo	Valoración económica del espacio utilizado para teletrabajo	100,00 €/mes
Registro de marca	Protección legal de la identidad corporativa del proyecto	150,45 €
Publicidad de lanzamiento	Promoción inicial para dar visibilidad al proyecto	250,00 €
Material de oficina	Papelería y material auxiliar	50,00 €

La utilización de herramientas gratuitas o de código abierto ha permitido reducir significativamente los costes del proyecto, manteniendo al mismo tiempo un entorno de trabajo profesional y adecuado para el desarrollo de la aplicación

4.2. Presupuesto económico

A partir del inventario de recursos materiales se realiza una estimación económica de los medios necesarios para la puesta en marcha y mantenimiento inicial del proyecto.

Inversión inicial

Concepto	Coste (€)
Ordenador portátil	1.180,00
Monitor adicional	180,00
Dominio web (primer año)	5,25
Hosting web (primer año)	93,00
Material de oficina	50,00
Registro de marca	150,45
Campaña publicitaria de lanzamiento	250,00
Fondo para imprevistos	341,30
TOTAL INVERSIÓN INICIAL	2.250,00

Gastos anuales previstos

Concepto	Importe anual (€)
Salario bruto del promotor	15.624,48
Seguridad Social empresa	4.999,80
Cuota de autónomos	1.063,68
Hosting y alojamiento web	93,00
Línea telefónica e internet	360,00
Electricidad y suministros	300,00
Utilización del espacio de trabajo	1.200,00
Gestoría	600,00
Publicidad online	1.200,00
Amortizaciones	340,00
TOTAL GASTOS ANUALES	25.780,96

Estos recursos constituyen la base material necesaria para garantizar el correcto funcionamiento de Inamania durante su fase de desarrollo y explotación. La inversión requerida es relativamente reducida en comparación con otros proyectos tecnológicos, debido al uso de herramientas gratuitas, software de código abierto y una infraestructura sencilla basada en servicios de alojamiento web convencionales.

5. Recursos humanos

Los recursos humanos constituyen uno de los elementos fundamentales para el desarrollo y mantenimiento de Inamania. Aunque inicialmente el proyecto será desarrollado y gestionado por una única persona, resulta necesario identificar las funciones necesarias para garantizar el correcto funcionamiento de la plataforma y valorar económicamente el trabajo realizado.

Durante las primeras fases del proyecto, todas las tareas serán asumidas por el promotor, quien ejercerá simultáneamente funciones de análisis, desarrollo, administración y gestión de contenidos. No obstante, se contempla la posibilidad de incorporar nuevos colaboradores en caso de crecimiento de la plataforma y aumento del número de usuarios.

5.1. Organización

La estructura organizativa de Inamania es sencilla debido a las dimensiones iniciales del proyecto. Al tratarse de una plataforma web desarrollada por una sola persona, todas las responsabilidades recaen inicialmente sobre el promotor.

Organización inicial

Puesto	Funciones
Desarrollador Full Stack	Desarrollo de la aplicación web, programación, diseño de base de datos y mantenimiento técnico
Administrador de contenidos	Gestión de publicaciones, eventos y contenidos de la plataforma
Administrador de sistemas	Gestión del hosting, copias de seguridad y seguridad de la aplicación
Responsable de comunidad	Supervisión de usuarios, comentarios y actividades de la comunidad

No obstante, con el crecimiento de la comunidad y el aumento de usuarios registrados, se prevé una estructura organizativa usando modelos compuesta por los siguientes perfiles:

- **Técnicos**
Responsable de la planificación del proyecto, desarrollo de nuevas funcionalidades, mantenimiento de la infraestructura tecnológica y administración de la base de datos.
- **Moderación**
Encargados de supervisar el contenido generado por los usuarios como los post y comentarios del foro, además deberán de gestionar el contenido de eventos y de la página web.
- **Soporte técnico**
Responsables de atender consultas de los usuarios, resolver incidencias técnicas y ofrecer asistencia relacionada con el uso de la aplicación, problemas de eventos y de pagos de estos.

Esta estructura permite una gestión eficiente de las diferentes áreas del proyecto y facilita su escalabilidad futura.

Profesiograma

Puesto

Desarrollador/a Web Full Stack

Descripción del puesto

Profesional encargado del desarrollo, mantenimiento y gestión de la plataforma web, así como de la administración de sus contenidos y funcionamiento técnico.

Funciones

- Desarrollo de la aplicación web
- Diseño y gestión de bases de datos
- Implementación de nuevas funcionalidades
- Mantenimiento y actualización de la plataforma
- Gestión de contenidos y usuarios
- Administración del hosting y sistema

Formación requerida

- Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Web (o similar)

Conocimientos técnicos

- Lenguajes: HTML, CSS, JavaScript
- Programación backend: PHP
- Bases de datos: MySQL
- Control de versiones: Git y GitHub
- Diseño web responsive (Bootstrap)

Habilidades y competencias

- Capacidad de organización
- Resolución de problemas
- Trabajo autónomo
- Adaptación a nuevas tecnologías
- Pensamiento lógico

Experiencia

- No imprescindible en la fase inicial
- Valorable experiencia previa en desarrollo web

Condiciones del puesto

- Trabajo en modalidad de teletrabajo
- Jornada flexible
- Actividad desarrollada como autónomo

5.2. Contratación

Dado que Inamania comienza como un proyecto de pequeño tamaño, no se prevé la contratación de personal durante el primer año de actividad.

Sin embargo, para realizar una valoración económica realista del proyecto, se considera el coste equivalente al trabajo realizado por el promotor.

Como referencia se utiliza un perfil profesional relacionado con el desarrollo de aplicaciones web y la administración de plataformas digitales.

Coste anual estimado

Concepto	Importe
Salario bruto anual estimado	15.624,48 €
Seguridad Social empresa	4.999,80 €
Coste anual total	20.624,28 €

Esta valoración permite reflejar adecuadamente los recursos humanos necesarios para el desarrollo y explotación de la plataforma, independientemente de que el trabajo sea realizado por el propio promotor.

Posibles contrataciones futuras

En caso de crecimiento de la plataforma podrían incorporarse los siguientes perfiles:

- Moderador de comunidad.
- Técnico de soporte.
- Colaborador para la organización de eventos.

5.3. Prevención de riesgos laborales

Durante el desarrollo y mantenimiento de INAMANIA pueden aparecer diversos riesgos laborales asociados principalmente al uso continuado de equipos informáticos y al trabajo prolongado frente a pantallas de visualización de datos.

Uno de los riesgos más frecuentes es la **fatiga visual**, que puede provocar cansancio ocular, visión borrosa o dolores de cabeza. Para minimizar estos efectos se realizarán descansos periódicos, se ajustará correctamente el brillo de las pantallas y se garantizará una iluminación adecuada del puesto de trabajo.

También existen **riesgos ergonómicos** derivados de mantener posturas inadecuadas durante largos periodos de tiempo. Estas situaciones pueden ocasionar molestias musculares, dolores cervicales o problemas lumbares. Para prevenirlos se utilizará mobiliario ergonómico, se mantendrá una postura correcta durante la jornada laboral y se realizarán ejercicios de estiramiento de forma periódica.

Por otra parte, el trabajo relacionado con el desarrollo de software puede generar **riesgos psicosociales**, como estrés, fatiga mental o disminución del rendimiento. Para reducir estos riesgos se planificarán adecuadamente las tareas, se establecerán objetivos realistas y se respetarán los periodos de descanso necesarios.

Asimismo, el uso de equipos electrónicos implica la existencia de **riesgos eléctricos**, que podrían provocar daños en los dispositivos o accidentes derivados de instalaciones defectuosas. Como medida preventiva se revisarán periódicamente los equipos y conexiones eléctricas, se utilizará material homologado y se evitarán sobrecargas en la instalación.

Finalmente, al tratarse de un proyecto basado en información digital, debe considerarse el riesgo de **pérdida de datos**, ya sea por fallos técnicos o errores humanos. Para minimizar este problema se realizarán copias de seguridad periódicas y se mantendrán actualizados los sistemas de protección y recuperación de información.

La aplicación de estas medidas permitirá desarrollar el proyecto en condiciones seguras, reduciendo los riesgos laborales y favoreciendo un entorno de trabajo adecuado para el desarrollo y mantenimiento de la plataforma.

6. Viabilidad técnica

Antes de llevar a cabo la implantación de Inamania es necesario analizar la viabilidad técnica del proyecto. Este estudio permite determinar si se dispone de los recursos tecnológicos, conocimientos y herramientas necesarios para desarrollar, desplegar y mantener la aplicación web de forma eficiente, segura y sostenible a lo largo del tiempo.

El análisis de viabilidad técnica resulta fundamental para garantizar que todas las funcionalidades previstas puedan implementarse correctamente utilizando tecnologías adecuadas y recursos accesibles, evitando limitaciones que comprometan el desarrollo o la evolución futura de la plataforma.

6.1. Estudio de viabilidad técnica

Tras analizar los requisitos y funcionalidades de Inamania, se concluye que el proyecto es técnicamente viable, ya que las tecnologías seleccionadas son adecuadas para su desarrollo y permiten implementar todas las características previstas de forma eficiente.

Los principales recursos técnicos utilizados son:

- HTML, CSS y JavaScript para el desarrollo de la interfaz de usuario.
- Bootstrap para facilitar el diseño responsive y la adaptación a distintos dispositivos.
- PHP para la programación del lado servidor y la gestión de la lógica de negocio.
- MySQL como sistema gestor de bases de datos.
- Git y GitHub para el control de versiones y la gestión del código fuente.
- Un servicio de hosting para el despliegue y publicación de la aplicación web.

Además, el desarrollo puede realizarse utilizando equipos informáticos convencionales, por lo que no es necesaria la adquisición de hardware especializado ni de infraestructuras complejas. Esto reduce significativamente las barreras técnicas para la puesta en marcha del proyecto.

Las funcionalidades planteadas, como la gestión de usuarios, publicaciones, comentarios, eventos y bases de datos relacionadas con los videojuegos, pueden desarrollarse mediante herramientas y metodologías ampliamente conocidas. Asimismo, la experiencia adquirida durante el ciclo formativo proporciona los conocimientos necesarios para afrontar el desarrollo de cada uno de estos componentes.

Por otro lado, la arquitectura de la aplicación ha sido diseñada de forma modular, lo que facilita tanto el mantenimiento como la incorporación de nuevas funcionalidades en el futuro. Esta estructura permitirá adaptar la plataforma al crecimiento de la comunidad sin necesidad de realizar cambios importantes en su funcionamiento.

Por todo ello, se concluye que Inamania es un proyecto técnicamente viable, ya que dispone de las herramientas, conocimientos y recursos necesarios para su desarrollo, implantación y mantenimiento.

7. Viabilidad económica-financiera

Antes de llevar a cabo la implantación de Inamania es necesario analizar la viabilidad económica y financiera del proyecto. Este estudio permite determinar la inversión necesaria para su puesta en marcha, las fuentes de financiación disponibles y la capacidad del proyecto para generar ingresos suficientes que permitan cubrir los costes de funcionamiento y garantizar su continuidad.

Elección de la forma jurídica

He decidido desarrollar la actividad como **empresario individual (autónomo)** porque es la forma jurídica que mejor se adapta a las características iniciales del proyecto.

En primer lugar, se trata de un proyecto que será gestionado por **una única persona**, lo que hace innecesaria la creación de una sociedad. Tal y como se recoge en el proyecto, todas las funciones (desarrollo, administración y gestión) son asumidas inicialmente por mi.

Además, el negocio requiere una **inversión inicial reducida**, aproximadamente 2.250 €, lo que hace más conveniente una estructura sencilla sin costes elevados de constitución. En este sentido, el régimen de autónomos permite iniciar la actividad de forma rápida y económica.

Otro aspecto clave es la **simplicidad en la gestión administrativa y fiscal**. Como autónomo, los trámites son más ágiles y se reducen a:

- Alta en Hacienda (IAE y declaración censal)
- Alta en la Seguridad Social (régimen de autónomos)
- Licencias municipales si fueran necesarias

Esto facilita la puesta en marcha del proyecto sin necesidad de procesos complejos.

Por otro lado, esta forma jurídica ofrece **flexibilidad**, permitiendo adaptar el negocio fácilmente en función de su evolución. De hecho, el propio proyecto contempla la posibilidad de crecer e incorporar personal en el futuro.

Finalmente, en el análisis económico se indica que esta opción es la más adecuada porque:

- Permite una **constitución rápida**.
- Tiene **menores costes administrativos**.
- Presenta una **estructura organizativa sencilla**.

7.1. Inversiones y gastos

Antes de llevar a cabo la implantación de Inamania es necesario analizar la viabilidad económica y financiera del proyecto. Este estudio permite determinar la inversión necesaria para su puesta en marcha, las fuentes de financiación disponibles y la capacidad del proyecto para generar ingresos suficientes que permitan cubrir los costes de funcionamiento y garantizar su continuidad, será importante tener en cuenta el desarrollo de la actividad es en teletrabajo por lo que añadiremos un gasto de 100 mensuales destinados a cubrir el uso de la vivienda como lugar de trabajo.

Inversiones iniciales

Concepto	Importe (€)
Ordenador portátil para desarrollo	1.180,00
Monitor adicional	180,00
Dominio web	5,25
Hosting anual	93,00
Material de oficina	50,00
Registro de marca	150,45
Campaña publicitaria de lanzamiento	250,00
Fondo de maniobra inicial	341,30
TOTAL INVERSIÓN INICIAL	2.250,00

Gastos anuales

Concepto	Importe anual (€)
Salario bruto del promotor	15.624,48
Seguridad social	4.999,8
Cuota de autónomos	1.063,68
Hosting y alojamiento web	93,00
Línea telefónica e internet	360,00
Electricidad y suministros	300,00
Utilización del espacio de trabajo (Teletrabajo)	1.200,00
Gestoría	600,00
Publicidad online	1.200,00
Amortizaciones	340
TOTAL GASTOS ANUALES	25.780,96

Gastos mensuales

Concepto	Gasto mensual (€)
Salario del promotor	1.302,04
Seguridad social	416,65
Cuota de autónomos	88,64
Hosting y alojamiento web	7,75
Internet y comunicaciones	30,00
Electricidad y suministros	25,00
Utilización del espacio de trabajo (teletrabajo)	100,00
Gestoría	50,00
Publicidad y marketing digital	100,00
Amortizaciones	28,33
Total gastos mensuales	2.148,41

Teniendo en cuenta estas previsiones, la inversión necesaria para iniciar la actividad es reducida de forma inicial pero abundante a la hora de mantenerlo en el tiempo debido a los gastos que se van constituyendo mes a mes, haciendo que haga falta financiación externa para poder afrontar el primer año.

7.2. Financiación

Una vez determinada la inversión necesaria para la puesta en marcha de Inamania, es necesario establecer las fuentes de financiación que permitirán cubrir tanto la inversión inicial como las necesidades económicas futuras derivadas del crecimiento y mantenimiento del proyecto.

La financiación se obtendrá mediante una combinación de recursos propios aportados por el promotor y financiación ajena a través de un préstamo bancario. Esta estrategia permite disponer de liquidez suficiente para garantizar el correcto funcionamiento de la plataforma durante sus primeros años de actividad, así como afrontar posibles imprevistos o inversiones futuras.

Fuentes de financiación

Fuente de financiación	Importe (€)
Aportación del promotor	10.000,00
Préstamo bancario	20.000,00
TOTAL FINANCIACIÓN	30.000,00

Coste del préstamo

Para complementar la aportación propia se solicitará un préstamo bancario de 20.000 € con un plazo de amortización de 5 años y un tipo de interés estimado del 6 % anual.

Año	Capital pendiente inicial (€)	Cuota anual (€)	Intereses (€)	Amortización (€)	Capital pendiente final (€)
1	20.000,00	4.747,93	1.200,00	3.547,93	16.452,07
2	16.452,07	4.747,93	987,12	3.760,80	12.691,27
3	12.691,27	4.747,93	761,48	3.986,45	8.704,82
4	8.704,82	4.747,93	522,29	4.225,64	4.479,18
5	4.479,18	4.747,93	268,75	4.479,18	0,00

El capital obtenido por la financiación ira repartida a diferentes sectores que son:

Destino de los fondos obtenidos

La mayor parte de los recursos financieros se destinará a garantizar la estabilidad económica del proyecto durante sus primeros años de funcionamiento. Esta planificación permite afrontar gastos imprevistos, realizar mejoras técnicas, ampliar servicios, desarrollar nuevas funcionalidades y mantener la actividad incluso en escenarios de ingresos inferiores a los inicialmente previstos.

La existencia de un fondo de maniobra proporciona una mayor capacidad de respuesta ante posibles incidencias y contribuye a reducir el riesgo financiero asociado a la fase inicial de implantación del proyecto.

7.3. Viabilidad económico-financiera

Una vez analizadas las inversiones necesarias y las fuentes de financiación disponibles, resulta necesario evaluar la viabilidad económica del proyecto mediante una previsión de ingresos, gastos y tesorería. Este análisis permite determinar si Inamania será capaz de generar recursos suficientes para mantener su actividad y garantizar su sostenibilidad a medio y largo plazo.

Evolución prevista de los ingresos

Durante los primeros meses de actividad, los ingresos serán reducidos debido a que la plataforma se encontrará en fase de lanzamiento y captación de usuarios. A medida que aumente el número de usuarios registrados, la actividad de la comunidad y la visibilidad del proyecto, se prevé un crecimiento progresivo de los ingresos procedentes de publicidad, cuentas premium y porcentajes de la gestión de los eventos.

Mes	Usuarios registrados estimados	Ingresos previstos (€)
Enero	50	800
Febrero	80	1.000
Marzo	120	1.300
Abril	180	1.600
Mayo	250	1.900
Junio	350	2.100
Julio	500	2.300
Agosto	650	2.500
Septiembre	800	2.700
Octubre	950	2.900
Noviembre	1.100	3.100
Diciembre	1.450	3.600
TOTAL	1.300 usuarios	25.900 €

Cuenta de Pérdidas y Ganancias prevista

Una vez que tenemos calculado los ingresos estimados a lo largo de los meses y los gastos tanto mensuales como anuales podemos realizar la viabilidad de las cuentas.

Ingresos de explotación

Concepto	Importe (€)
Ingresos por publicidad	8.000,00
Ingresos por cuentas premium	15.900,00
Ingresos por colaboraciones y eventos	2.000,00
TOTAL INGRESOS DE EXPLOTACIÓN	25.900,00

Gastos de explotación

Concepto	Importe (€)
Nómina del promotor	15.624,48
Seguridad Social empresa	4.999,80
Cuota de autónomos	1.063,68
Hosting y alojamiento web	93,00
Internet y comunicaciones	360,00
Electricidad y suministros	300,00
Utilización espacio de trabajo (teletrabajo)	1.200,00
Servicios de gestoría	600,00
Publicidad y marketing digital	1.200,00
Amortizaciones	340,00
TOTAL GASTOS DE EXPLOTACIÓN	25.780,96

Resultado de explotación

Concepto	Importe (€)
Total ingresos de explotación	25.900,00
Total gastos de explotación	-25.780,96
RESULTADO DE EXPLOTACIÓN	119,04

Gastos financieros

Concepto	Importe (€)
Intereses del préstamo bancario (año 1)	1.200,00
TOTAL GASTOS FINANCIEROS	1.200,00

Resultado primer año

Concepto	Importe (€)
Resultado de explotación	119,04
Resultado financiero	-1.200,00
RESULTADO ANTES DE IMPUESTOS	-1.080,96

Los impuestos principales son el IVA y el IRPF. El IVA no se incluye en la Cuenta de Pérdidas y Ganancias porque la empresa únicamente lo recauda y posteriormente lo liquida con Hacienda, por lo que no constituye ni un ingreso ni un gasto.

Respecto al IRPF, este impuesto se calcula sobre el beneficio obtenido por la actividad económica. En la previsión realizada para el primer ejercicio, el resultado antes de impuestos es de **-1.080,96 €**, por lo que no existe beneficio fiscal. Al no generarse una base imponible positiva, no corresponde pagar IRPF sobre la actividad durante este ejercicio, motivo por el cual el gasto por impuestos es **0 €**.

Plan de Tesorería

El plan de tesorería permite analizar la liquidez del proyecto, es decir, la capacidad de la empresa para hacer frente a sus pagos a corto plazo. A continuación, se presenta la previsión mensual de cobros y pagos durante el primer año de actividad.

Para su elaboración se han tenido en cuenta:

- Tesorería inicial: **27.750 €**
- Cobros mensuales: los ingresos previstos.
- Pagos mensuales: **2.544,07 €** (2.148,41 € de gastos + 395,66 € de cuota del préstamo).

Concepto	Ene	Feb	Mar	Abr	May	Jun	Jul	Ago	Sep	Oct	Nov	Dic
Saldo inicial mes	27.750,00	26.005,93	24.461,86	23.217,79	22.273,72	21.629,65	21.185,58	20.941,51	20.897,44	21.053,37	21.409,30	21.965,23
Cobros	800,00	1.000,00	1.300,00	1.600,00	1.900,00	2.100,00	2.300,00	2.500,00	2.700,00	2.900,00	3.100,00	3.600,00
Publicidad	250,00	350,00	500,00	600,00	700,00	700,00	750,00	800,00	850,00	900,00	1.000,00	1.600,00
Cuentas premium	500,00	600,00	700,00	900,00	1.100,00	1.300,00	1.450,00	1.600,00	1.750,00	1.900,00	2.000,00	2.100,00
Eventos y colaboraciones	50,00	50,00	100,00	100,00	100,00	100,00	100,00	100,00	100,00	100,00	100,00	200,00
Pagos	2.544,07	2.544,07	2.544,07	2.544,07	2.544,07	2.544,07	2.544,07	2.544,07	2.544,07	2.544,07	2.544,07	2.544,07
Nómina promotor	1.302,04	1.302,04	1.302,04	1.302,04	1.302,04	1.302,04	1.302,04	1.302,04	1.302,04	1.302,04	1.302,04	1.302,04
Seguridad Social	416,65	416,65	416,65	416,65	416,65	416,65	416,65	416,65	416,65	416,65	416,65	416,65
Autónomos	88,64	88,64	88,64	88,64	88,64	88,64	88,64	88,64	88,64	88,64	88,64	88,64
Hosting	7,75	7,75	7,75	7,75	7,75	7,75	7,75	7,75	7,75	7,75	7,75	7,75
Internet	30,00	30,00	30,00	30,00	30,00	30,00	30,00	30,00	30,00	30,00	30,00	30,00
Electricidad	25,00	25,00	25,00	25,00	25,00	25,00	25,00	25,00	25,00	25,00	25,00	25,00
Espacio de trabajo	100,00	100,00	100,00	100,00	100,00	100,00	100,00	100,00	100,00	100,00	100,00	100,00
Gestoría	50,00	50,00	50,00	50,00	50,00	50,00	50,00	50,00	50,00	50,00	50,00	50,00
Publicidad	100,00	100,00	100,00	100,00	100,00	100,00	100,00	100,00	100,00	100,00	100,00	100,00
Amortización	28,33	28,33	28,33	28,33	28,33	28,33	28,33	28,33	28,33	28,33	28,33	28,33
Cuota préstamo	395,66	395,66	395,66	395,66	395,66	395,66	395,66	395,66	395,66	395,66	395,66	395,66
Saldo final mes	26.005,93	24.461,86	23.217,79	22.273,72	21.629,65	21.185,58	20.941,51	20.897,44	21.053,37	21.409,30	21.965,23	23.021,16

Balance de Situación del 1º año

Activo

ACTIVO	Importe (€)
Activo no corriente	
Equipos informáticos (portátil y monitor)	1.020,00
Dominio web y alojamiento	98,25
Total Activo No Corriente	1.118,25
Activo corriente	
Tesorería y bancos	23.021,16
Total Activo Corriente	23.021,16
TOTAL ACTIVO	24.139,41

Patrimonio Neto y Pasivo

PATRIMONIO NETO Y PASIVO	Importe (€)
Patrimonio Neto	
Aportación inicial del promotor	10.000,00
Resultado del ejercicio	-1.080,96
Total Patrimonio Neto	8.919,04
Pasivo no corriente	
Préstamo bancario a largo plazo	11.459,57
Total Pasivo No Corriente	11.459,57
Pasivo corriente	
Deuda préstamo corto plazo (siguiente anualidad)	3.760,80
Total Pasivo Corriente	3.760,80
TOTAL PATRIMONIO NETO Y PASIVO	24.139,41

Impuestos y amortizaciones

Como empresario individual, el promotor deberá cumplir con las obligaciones fiscales correspondientes a la actividad económica desarrollada, principalmente mediante la liquidación del IVA y del IRPF según la normativa vigente.

Asimismo, los equipos informáticos utilizados para el desarrollo de la actividad estarán sujetos a amortización contable, distribuyendo su coste a lo largo de su vida útil estimada.

Equipo	Valor (€)	Vida útil	Amortización anual
Portátil	1.180	4 años	295 €
Monitor	180	4 años	45 €
Total amortización anual			340 €

Conclusión

A pesar de que la cuenta de pérdidas ganancias presenta un resultado ligeramente negativo debido a los costes de puesta en marcha y consolidación del proyecto, la evolución prevista de los ingresos muestra una tendencia claramente creciente. Gracias a la financiación inicial obtenida y al fondo de maniobra disponible, Inamania dispone de recursos suficientes para afrontar esta fase inicial y alcanzar la rentabilidad durante los ejercicios posteriores, por lo que el proyecto puede considerarse económicamente viable.

8. Conclusión

El desarrollo de Inamania ha permitido crear una plataforma web moderna, funcional y orientada a una comunidad concreta de usuarios apasionados por la saga Inazuma Eleven. A lo largo del proyecto se han aplicado conocimientos relacionados con el desarrollo web, diseño de bases de datos, programación backend y frontend, accesibilidad, usabilidad y seguridad informática, integrando todas estas áreas en una única aplicación completa y operativa.

Uno de los principales objetivos del proyecto era construir un entorno donde los usuarios pudieran consultar información, interactuar con otros miembros de la comunidad y participar en eventos relacionados con la franquicia. Este objetivo se ha cumplido mediante la implementación de diferentes funcionalidades como foros, sistemas de búsqueda dinámica, perfiles de usuario, gestión de eventos y paneles administrativos.

Además, durante el desarrollo se ha priorizado ofrecer una experiencia intuitiva, accesible y adaptable a cualquier dispositivo, aplicando buenas prácticas de diseño responsive y organización de contenidos. También se ha trabajado en mantener una estructura modular y escalable que facilite futuras ampliaciones y el mantenimiento del sistema.

Desde el punto de vista técnico y económico, el proyecto resulta viable gracias al uso de tecnologías gratuitas, ampliamente utilizadas y compatibles entre sí, lo que permite reducir costes sin limitar funcionalidades. Asimismo, la arquitectura desarrollada permite incorporar nuevas características en el futuro, como sistemas competitivos, nuevas guías, chat en tiempo real o integración con APIs externas.

En conclusión, Inamania no solo representa una aplicación web funcional, sino también un proyecto que demuestra la capacidad de diseñar, desarrollar y mantener una plataforma completa orientada a usuarios reales, combinando aspectos técnicos, visuales y organizativos de forma eficiente.

Bibliografía/Webgrafía

1. <https://www.level5.co.jp/vision2023/en/>
2. <https://fonts.google.com/specimen/Montserrat>
3. https://www.guiasnintendo.com/0_NINTENDO_DS/inazuma_eleven/inazuma_eleven_sp/explicacion.html
4. <https://fontawesome.com>
5. <https://getbootstrap.com>

ANEXOS

1. <https://inamania.es/>
2. <https://github.com/martineznavarroivan26/Proyecto-intermodular>
3. <https://github.com/martineznavarroivan26/Proyecto-intermodular/blob/main/config/BD.sql>
4. https://github.com/martineznavarroivan26/Proyecto-intermodular/blob/main/docs/Manual_Usuario_Inamania_v1_0.pdf